Blockchain-Assisted Lightweight Broadcast Encryption With Revocation Support for Secure ADS-B in IoT Aviation

Xuejun Zhang, *Member, IEEE*, Chong Yao, Yizhong Liu, *Member, IEEE*, Boyu Zhao, Cheng Chi, and Haohua Du, *Member, IEEE*

***Abstract*—Automatic dependent surveillance–broadcast (ADS-B) is widely deployed in both civil and uncrewed aviation networks, yet its plaintext broadcast design leaves it vulnerable to eavesdropping, spoofing, and message forgery. Existing cryptographic solutions incur excessive overhead, depend on complex key infrastructures, and fail to accommodate the broadcast nature of ADS-B, making them unsuitable for real-time internet of things (IoT) aviation scenarios. In this article, we propose ADSB-identity-based broadcast encryption (IBBE), the first lightweight and scalable security scheme tailored for confidential ADS-B broadcast communication in IoT-enabled aerial networks. ADSB-IBBE integrates a novel IBBE construction to enable efficient key distribution, together with a purpose-built, format-preserving stream cipher that secures critical ADS-B fields without extending the message size. It further incorporates a customized sharding consortium blockchain for decentralized identity management and rapid key revocation, while a header compression mechanism reduces transmission overhead. Security analysis confirms indistinguishability against selective-identity chosen ciphertext attacks (IND-sID-CCAs) security under the random oracle model (ROM). Experiments show over 80% lower computational cost and 90% reduced communication overhead compared to the most demanding baseline, with increasing advantages as the number of receivers grows, achieving the lowest overhead of 510-bit header and 80-bit ciphertext. These results demonstrate the suitability of our scheme for secure and efficient ADS-B communication in next-generation IoT-enabled air traffic management (ATM) systems.**

***Index Terms*—Air traffic management (ATM), automatic dependent surveillance–broadcast (ADS-B), identity-based broadcast encryption (IBBE), internet of things (IoT) aviation, sharding blockchain, stream cipher.**

NOMENCLATURE

$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$	Finite elliptic curve point group, $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$.
g, h	Generator of group $\mathbb{G}_1, \mathbb{G}_2$.
$H_1(\cdot)$	Hash function $H_1 : \{0,1\}^* \to \mathbb{Z}_p^*$.
$H_2(\cdot)$	Hash function $H_2 : \mathbb{G}_1 \to \mathbb{Z}_p^*$.
$\mathcal{L}, \mathcal{RL}$	Private key list and revocation list.
$\mathsf{MPK}, \mathsf{MSK}$	Main public key and main secret key.
ID_i	Identity of user i.
S	Identity set of ID_i, for $i = 1, 2, \ldots, s$.
$\mathsf{SK}_{\mathsf{ID}i}$	Private key of user i.
$\mathsf{Hdr}, \mathsf{Hdr}'$	Message header and compressed message header.
K	Symmetric key.
Ψ_v	Pseudorandom function.
m	ADS-B message (ME), $m \in \{0,1\}^*$.
IV	Initialization vector.
ts	Send or receive timestamp.
T_{bp}	Bilinear pairing map e: $\mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$.
T_{et}	Exponentiation in bilinear groups.
T_{sm_1}, T_{sm_2}	Scalar multiplication in G_1 and G_2.
$T_{pa_1}, T_{pa_2}, T_{pa_T}$	Group addition in G_1, G_2, and G_T.
T_{hash}	Hash computation.
$T_{\oplus}$	Bitwise XOR operation.
T_{aes}	AES encryption/decryption.
T_{stream}	Key stream generation operation.

I. INTRODUCTION

WITH the continuous growth of the global aviation industry, the complexity of air traffic management (ATM) [1] is steadily increasing, placing higher demands on efficient, secure, and real-time aircraft surveillance technologies. According to the 2024 fleet forecast released by Cirium [2], as of Q4 2024, the global active commercial fleet has reached 26 100 aircraft. Around 45 900 new aircraft are expected to be delivered by 2043 to meet the growing needs of global air transport. At the same time, the uncrewed aerial vehicle (UAV) market, including electric vertical takeoff and landing (eVTOL) aircraft, is experiencing rapid expansion. According to the Federal Aviation Administration (FAA) [3], by the end of 2023, over 842 000 UAVs had been registered in the United States alone, and the number of commercial uncrewed aircraft systems (UASs) is projected to exceed 1.12 million by 2028. This surge in aerial vehicle

Received 17 September 2025; revised 28 October 2025, 8 December 2025, and 31 December 2025; accepted 11 January 2026. Date of publication 15 January 2026; date of current version 26 March 2026. This work was supported in part by the Key Technology Research and Development of Low-altitude Intelligent Network under Grant 2024SGJ017 and in part by Zhejiang Provincial Natural Science Foundation of China under Grant LMS25F020014. *(Corresponding authors: Yizhong Liu; Haohua Du.)*

Xuejun Zhang and Chong Yao are with the Hefei Innovation Research Institute, Beihang University, Beijing 100191, China, also with the School of Electronic Information Engineering, Beihang University, Beijing 100191, China, and also with the Beijing Key Management Laboratory for Network-based Cooperative Air Traffic Management and the State Key Laboratory of CNS/ATM, Beijing 100191, China (e-mail: zhxj@buaa.edu.cn; yaochong@buaa.edu.cn).

Yizhong Liu, Boyu Zhao, and Haohua Du are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: liuyizhong@buaa.edu.cn; boyuzhao@buaa.edu.cn; duhaohua@buaa.edu.cn).

Cheng Chi is with the Research Institute of Industrial Internet of Things, China Academy of Information and Communications Technology, Beijing 100191, China (e-mail: chicheng@caict.ac.cn).

Digital Object Identifier 10.1109/JIOT.2026.3654126

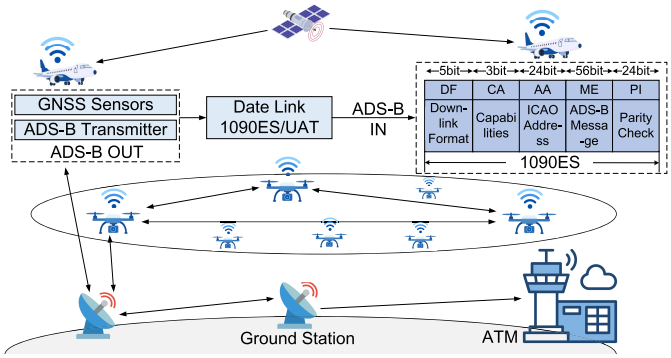

Fig. 1. ADS-B system architecture for aerial vehicles.

volume places increasing pressure on airspace management and necessitates the adoption of more advanced surveillance technologies.

ADS-B [4] is currently the most advanced Global Navigation Satellite System (GNSS)-based broadcast surveillance technology, widely adopted for both air-to-ground (A2G) and air-to-air (A2A) monitoring of crewed and uncrewed aircraft, as depicted in Fig. 1. Relying on GNSS, ADS-B provides high-precision flight data including position, velocity, and heading. Through ADS-B OUT, aircraft automatically broadcast their status information to ground stations and nearby aircraft, while receivers equipped with ADS-B IN can process the data in real time to enhance situational awareness. ADS-B primarily operates on two datalinks: 1090ES (commonly used) and 978-MHz UAT. Compared with traditional radar systems, ADS-B offers superior accuracy, faster update rates, and broader coverage, including in remote areas. As a result, it has become a core technology for global airspace awareness and ATM surveillance and has been mandated in multiple countries and regions. Meanwhile, the integration of ADS-B with internet of things (IoT)-enabled platforms, UAVs, and intelligent ground systems is accelerating the evolution of next-generation air traffic infrastructures, forming a key pillar of IoT-enabled ATM.

A. Technical Characteristics of ADS-B Systems

First, unidirectionality and broadcast property. ADS-B messages ME are transmitted by the sender in a broadcast format, allowing any receiver within a defined range to receive the messages. Second, short message length and high transmission frequency. Each ME consists of 112 bits, and approximately six messages are sent every second. Third, massive scale. The future airspace will involve a massive volume of aerial vehicles equipped with ADS-B systems, requiring the handling of a large volume of messages. In addition, ADS-B systems offer several notable advantages, including wide-area surveillance coverage, improved trajectory accuracy, and reduced infrastructure costs, making them a sustainable solution for air traffic surveillance [5]. These characteristics make ADS-B both advantageous and challenging to secure in future IoT-enabled air traffic environments.

B. Security Challenges of ADS-B Systems

Despite the operational benefits of ADS-B systems, they suffer from critical security vulnerabilities. In the absence of encryption and authentication mechanisms [6], ME are broadcast in plaintext and are easily subject to eavesdropping, forgery, and manipulation [7], [8]. Although various encryption schemes have been proposed to enhance ADS-B security, they remain limited in multiple aspects [9].

First, encryption overhead in large-scale deployments. Single encryption (SE) induces high computational and bandwidth costs, making it impractical for real-time, large-scale aviation environments. Broadcast encryption (BE) reduces redundancy but introduces complexity in key revocation, identity management, and dynamic receiver updates.

Second, centralized key management bottlenecks. Most existing schemes rely on centralized trust infrastructures, which introduce single points of failure (SPOF) and are ill-suited for decentralized identity control or responsive key revocation in dynamic airspace settings [10], [11], [12].

Third, incompatibility with ADS-B constraints. Existing identity-based broadcast encryption (IBBE) schemes are typically designed for general-purpose networks and disregard ADS-B-specific constraints such as strict latency, constrained message length, and limited on-board computational capacity. Their large ciphertexts, oversized headers, and computational inefficiencies render them unsuitable for ADS-B deployment.

Overall, existing encryption approaches for ADS-B remain inadequate. SE schemes lack scalability, BE schemes complicate dynamic revocation and identity-level control, and current IBBE constructions are too heavy for short messages and real-time operation. Unlike previous centralized or computation-intensive designs, we innovatively propose a lightweight, blockchain-assisted, and identity-driven BE framework that achieves efficient revocation while providing secure and scalable protection tailored for ADS-B systems.

C. Contributions

To address the gap between existing cryptographic methods and the strict performance demands of ADS-B in IoT-enabled aerial systems, we propose ADSB-IBBE, a lightweight identity-based BE scheme integrated with a sharding blockchain. The design enables decentralized identity management, efficient key revocation, and format-preserving message encryption, while ensuring strong confidentiality and low computational and communication overhead.

1) A lightweight and revocable BE scheme for ADS-B communication. We propose ADSB-IBBE, a lightweight IBBE scheme specifically designed for secure and scalable ADS-B communication in large-scale IoT-enabled aerial networks. It supports large-scale multireceiver environments by enabling authorized receivers to independently derive a common symmetric key. A header compression mechanism minimizes ciphertext redundancy, and a dynamic revocation strategy preserves broadcast confidentiality in dynamic receiver settings.
2) A blockchain-assisted framework for decentralized identity and key management. Leveraging a sharding consortium blockchain, the framework supports secure storage of flight plans and identity sets, timely key revocation, and efficient cross-shard synchronization. A sharding Byzantine fault-tolerant (BFT) consensus

ensures scalability, fault tolerance, and eliminates SPOF. Blockchain-assisted preflight identity prefetching avoids real-time in-flight queries, while dynamic update and revocation ensure that illegal users cannot decrypt broadcast messages.

3) A format-preserving stream cipher scheme for ME encryption. To ensure compatibility with the fixed-length ADS-B structure, we design a stream cipher that encrypts the ICAO address (AA) and ME fields without altering their format or length. The ciphertext matches the plaintext in size and is directly embedded into the original message without additional overhead. This protects aerial vehicles' identity and flight data privacy, and ensures seamless integration and efficient processing.
4) *Formal Security Proof and Empirical Evaluation:* We provide a rigorous proof that ADSB-IBBE achieves indistinguishability against selective-identity chosen ciphertext attack (IND-sID-CCA) security under the random oracle model (ROM), establishing provable confidentiality guarantees for BE. Experimental results show that as the number of receivers increases, the proposed scheme reduces overhead by over 80% compared to the baseline, with the lowest overhead of 510-bit header. This demonstrates the best overall trade-off between minimal communication cost and high computational efficiency.

D. Related Works

1) SE Schemes: To enhance the security [21], [22], [23], [24] of ADS-B, many existing approaches adopt SE, where the broadcaster encrypts each message separately for every intended receiver. Baek et al. [13] proposed a staged identity-based encryption (SIBE) scheme that enables frequent encryption and flexible key management, aiming to meet the efficiency demands of ADS-B in resource-constrained environments. Braeken [14] presented a holistic protection mechanism using the elliptic curve Qu Vanstone (ECQV)-based implicit certificates to ensure confidentiality and authentication. Yang et al. [25] designed a lightweight encryption scheme that preserves compatibility with the existing ADS-B protocol while enhancing data privacy and integrity. Zeng [26] proposed an ADS-B protection scheme based on dynamic order-preserving encryption (DOPE), which enables time-ordered queries over encrypted data and supports integrity verification. Burfeind et al. [27] investigated a hybrid approach combining format-preserving encryption (FPE) with asymmetric encryption to enhance message confidentiality while maintaining compatibility with the ADS-B protocol. However, SE-based schemes inherently lack scalability. Per-recipient encryption leads to linear computational and communication overhead, making it ill-suited for the real-time and resource-constrained nature of ADS-B. Moreover, SE schemes offer limited resistance to replay and ciphertext manipulation attacks in broadcast scenarios, and their reliance on public key infrastructure (PKI) complicates key distribution and revocation.

2) Be Schemes: To address the overhead of SE in multireceiver scenarios, Fiat and Naor [28] introduced BE, allowing a single ciphertext to be decrypted by multiple authorized users. Boneh et al. [29] proposed public-key BE schemes with reduced ciphertext size and key storage under collusion resistance. Guo et al. [30] constructed an adaptively secure scheme with constant-size ciphertexts using composite-order multilinear maps. Agrawal et al. [31] developed a learning with errors (LWE)-based BE scheme with formal security proofs in the standard model. Dupin and Abelard [32] proposed a symmetric BE scheme, named ΣΠBE, based on Boolean function decomposition to reduce bandwidth and storage overhead. However, traditional BE schemes [19], [20] are typically based on user indexing rather than user identity, which limits support for fine-grained, identity-based access control and weakens traceability. In dynamic aviation environments, this design hinders secure key revocation and exposes the system to unauthorized access if user-index mappings are compromised. Additionally, public-key BE still incurs high computational costs and long ciphertexts in large-scale deployments.

3) IBBE Schemes: To overcome the limitations of traditional BE, Delerablée [33] proposed the first IBBE scheme, extending identity-based encryption (IBE) to the broadcast setting. In IBE schemes [34], [35], users' identities serve as their public keys, simplifying key management. IBBE enables the sender to encrypt a message once and broadcast it to multiple authorized recipients, each of whom can decrypt it using their private key. Gentry and Waters [36] presented an adaptively secure IBBE scheme in the standard model with sublinear ciphertext size. Kim et al. [37] improved security guarantees using dual system encryption techniques. Ge and Wei [15] proposed a revocable IBBE using a binary-tree key structure, enabling efficient key updates for dynamic user sets. Ramanna [16] introduced a revocable IBBE scheme using inner-product encryption (IPE) and quasi-adaptive noninteractive zero-knowledge (QA-NIZK) proofs, achieving adaptive security under the standard model. Beyond theoretical developments, IBBE has also been explored in various application domains, such as cloud computing, vehicular networks (VANETs), and healthcare. Chen et al. [17] combined IBE and PKI to address key escrow and achieve CCA security in cloud storage. Deng et al. [18] proposed a certificateless multireceiver encryption scheme suitable for smart communities. Mandal [38] introduced an anonymous attribute-based BE for telemedicine, optimizing key size and decryption efficiency. In VANETs, Zhang et al. [39] presented a proxy re-encryption approach supporting identity-based broadcast sharing, while Zhong et al. [40] designed a fixed-length ciphertext IBBE scheme for vehicle-to-infrastructure (V2I) communication with improved efficiency via proxy-aided encryption. However, despite these advances, existing IBBE schemes still face challenges such as large broadcast headers, high computational costs, and the lack of CCA security, which make them vulnerable to ciphertext substitution and malleability attacks. Moreover, most existing schemes depend on centralized key management and are designed for internet-based infrastructures, lacking decentralized identity control and aviation-specific optimization.

TABLE I
COMPARISON WITH OTHER ENCRYPTION SCHEMES

Scheme	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	Ours
Encryption Type	SE	SE	RIBBE	IPE-IBBE	CBBE	CLMRE	MA-ABBE	IBE	IBBE
Lightweight Design	✗	✗	✗	✗	✗	✗	✗	✗	✓
Key Management	PKG	TTP	PKG	KeyGen	CA	KGC	KGA	KGC	KGC
Key Update/Revocation	✗	✗	BT	✗	✗	✗	Direct	Direct	PRF Ψ
Identity Management	Single Point	Single Point	Single Point	Single Point	Single Point	Single Point	Single Point	Single Point	Blockchain
Transmission Overhead	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ Long	$\mathcal{O}(1)$ Long	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ Short
Hardness Assumption	BDH	ECDLP	SXDH	SXDH	q-ABDHE & 1-BDHI	DBDH	GDDHE	q-PBDHE	GDDHE
Proof Model	ROM	—	Standard	Standard	Standard	Standard	Standard	Standard	ROM
Security Level	CPA	—	CPA	CPA	CCA	CCA	CPA	CPA	CCA
Decentralization	✗	✗	✗	✗	✗	✗	✗	✗	✓
Packet Loss Tolerance	✗	✗	✗	✗	✗	✗	✗	✗	RS
Application Scenario	ADS-B (small-scale)	ADS-B (small-scale)	Conditionally	Conditionally	Cloud Storage	Smart Community	Fog computing	Medical IOT	ADS-B (large-scale)

Note: n is the number of receivers. CPA = Chosen Plaintext Attack; BDH = Bilinear Diffie-Hellman assumption; ECDLP = Elliptic Curve Discrete Logarithm Problem; SXDH = Symmetric eXternal Diffie-Hellman assumption; q-ABDHE/q-PBDHE = q-augmented/parallel Bilinear Diffie-Hellman Exponent; 1-BDHI = 1-Bilinear Diffie-Hellman Inversion; DBDH = Decisional Bilinear Diffie-Hellman.

These limitations are particularly problematic for real-time, bandwidth-constrained ADS-B systems, making current IBBE constructions impractical for deployment.

E. Comparison

We compare various encryption schemes in terms of their suitability for ADS-B systems, as summarized in Table I. Compared with prior work, our scheme achieves lightweight design, efficient key revocation, decentralized identity management, and short ciphertexts, making it well-suited for resource-constrained ADS-B environments.

F. Article Organization

The remainder of this article is organized as follows. Section II presents the preliminaries and fundamental definitions. Section III introduces the system model and security goals. Section IV details the construction of the proposed ADSB-IBBE scheme, including the blockchain-based identity and key management framework and the stream-cipher-based encryption mechanism. Section V provides a comprehensive performance analysis and experimental evaluation. Section VI concludes the article and discusses future work. Appendix A gives the formal security proof, and Appendix B presents the system-level security analysis.

II. PRELIMINARIES

This section introduces the background knowledge of our scheme.

A. ME Properties

ME consist of multiple fields [41], each carrying specific information to assist receivers in parsing and utilizing the data. The DF field denotes the message type and format, aiding receivers in identifying and parsing the data. For instance, DF = 17 indicates a standard ME, DF = 18 is used for extended, customizable ME, and DF = 19 pertains to military or national use. The CA field indicates the aerial vehicles' capabilities and functionalities, such as support for ADS-B In and Out. The AA field includes unique identification and recognition information for the aerial vehicles, such as the International Civil Aviation Organization (ICAO) 24-bit address. The ME field carries detailed flight data, such as position and speed, forming the core part of ME. The PI field is used to describe the accuracy and reliability of the position data. Through these detailed data fields, the ADS-B system provides precise real-time aerial vehicles' position information and flight dynamics, thereby enhancing flight safety and airspace utilization efficiency.

B. Bilinear Pairing

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be additive cyclic groups and $\mathbb{G}_T$ a multiplicative cyclic group, all of prime order p. A bilinear pairing is a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ satisfying the following properties.

1) *Bilinearity:* $\forall a, b \in \mathbb{Z}_p^*$, $\forall P \in \mathbb{G}_1, Q \in \mathbb{G}_2$, $e(aP, bQ) = e(P, Q)^{ab}$.
2) *Nondegeneracy:* $\exists P \in \mathbb{G}_1, Q \in \mathbb{G}_2$ such that $e(P, Q) \neq 1$.
3) *Computability:* $\forall P \in \mathbb{G}_1, Q \in \mathbb{G}_2$, $e(P, Q)$ can be efficiently computed in polynomial time.

C. Standard IBBE Model

A standard IBBE scheme typically consists of the following four polynomial-time algorithms.

1) *Setup(λ, l):* Given a security parameter λ and the maximum number of users l, the key generation center (KGC) outputs the MSK and the (MPK).
2) *KeyGen($\mathsf{MSK}, \mathsf{ID}_i$):* Given the MSK and a user identity ID_i, the PKG generates and outputs the corresponding private key SK_i.
3) *Encrypt(MPK, S, K):* Given the MPK, a recipient set $S \subseteq \{\mathsf{ID}_1, \ldots, \mathsf{ID}_l\}$, and a symmetric key K, the sender generates a ciphertext header Hdr such that all users in S can recover the key K using their private keys.
4) *Decrypt($\mathsf{SK}_i, \mathsf{Hdr}, \mathsf{MPK}$):* Given a ciphertext header Hdr, the public key MPK, and a private key SK_i corresponding to identity ID_i, the receiver recovers the symmetric key K if $\mathsf{ID}_i \in S$; otherwise, it returns $\perp$.

D. Security Model

For any polynomial-time adversary $\mathcal{A}$ and simulator $\mathcal{B}$, the $\mathsf{IND\text{-}sID\text{-}CCA}$ security model is defined by the following game.

1) *Initialization:* The adversary $\mathcal{A}$ outputs a challenge identity set $S^* = \{\mathsf{ID}_1^*, \mathsf{ID}_2^*, \ldots, \mathsf{ID}_{s^*}^*\}$, where $s^* \leq l$.
2) *System Setup:* The simulator $\mathcal{B}$ generates the system MPK and sends it to $\mathcal{A}$, where $\mathsf{MPK} \leftarrow \text{Setup}(\lambda, l)$.
3) *Hash Queries:* The adversary $\mathcal{A}$ can query the hash functions H_1 or H_2, and the simulator $\mathcal{B}$ returns the corresponding hash value.
4) *Query Phase 1:* The adversary $\mathcal{A}$ can adaptively issue the following queries.
   a) *Extraction Query:* The adversary $\mathcal{A}$ queries for the private key corresponding to an identity $\mathsf{ID}_i \notin S^*$. The simulator $\mathcal{B}$ returns the private key $\mathsf{SK}_{\mathsf{ID}_i}$, where $\mathsf{SK}_{\mathsf{ID}_i} \leftarrow \text{KeyGen}(\mathsf{MSK}, \mathsf{ID}_i)$.
   b) *Decryption Query:* The adversary $\mathcal{A}$ queries the decryption of ciphertext Hdr for $S \subseteq S^*$, $\mathsf{ID}_i \in S$. The simulator $\mathcal{B}$ decrypts the ciphertext and returns the result.
5) *Challenge Phase:* After Query Phase 1, simulator $\mathcal{B}$ runs $\text{Encrypt}(S^*, \mathsf{MPK})$ to generate the challenge ciphertext Hdr^* and key K^*. It randomly selects $\mu \leftarrow_R \{0, 1\}$, sets $K_\mu = K^*$, and chooses $K_{1-\mu}$ randomly, then sends $(\mathsf{Hdr}^*, K_0, K_1)$ to $\mathcal{A}$.
6) *Query Phase 2:* The adversary $\mathcal{A}$ continues issuing queries but cannot query the decryption of the ciphertext Hdr^*.
7) *Guess Phase:* Finally, the adversary $\mathcal{A}$ outputs a guess $\mu' \in \{0, 1\}$. If $\mu' = \mu$, then $\mathcal{A}$ wins the game. The advantage of $\mathcal{A}$ is given by

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND-sID-CCA}}(\lambda) = \left| \Pr[\mu' = \mu] - \frac{1}{2} \right|. \tag{1}$$

Theorem 1: In the IND-sID-CCA security model, if the advantage $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND-sID-CCA}}(\lambda)$ is negligible for any adversary $\mathcal{A}$, then the scheme is considered IND-sID-CCA secure.

E. Byzantine Fault Tolerance (BFT)

BFT is a consensus mechanism used in blockchain systems to ensure agreement even in the presence of faulty or malicious nodes, maintaining the network's security and stability. Compared to traditional methods like proof of work (PoW) and proof of stake (PoS), BFT offers higher throughput and lower latency, making it ideal for real-time applications.

BFT works by allowing nodes to exchange messages and validations, reaching consensus despite some nodes failing, as long as the number of faulty nodes F does not exceed a threshold. The system requires at least $N \geq 3F + 1$ nodes to ensure reliable consensus, even with faulty nodes. This decentralized approach, which does not require synchronization between all nodes, makes BFT particularly effective in blockchain environments.

F. Stream Cipher

Stream ciphers are encryption algorithms that encrypt each bit or byte of plaintext with a corresponding bit or byte from a random key stream, providing efficient and lightweight encryption suitable for performance-critical applications.

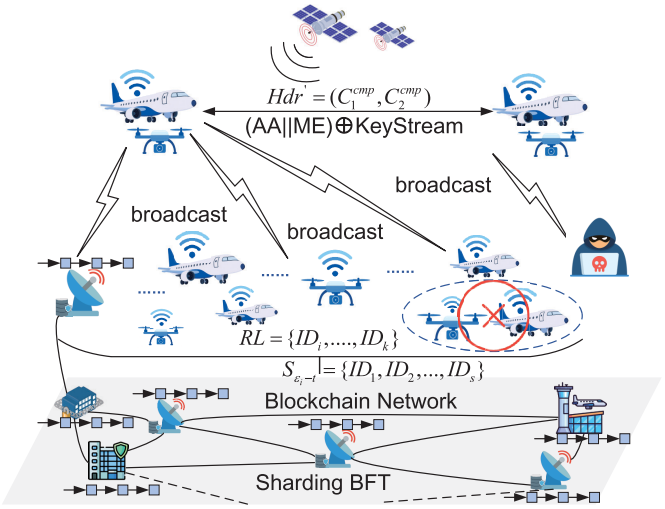

Fig. 2. Overview of our blockchain-based ADS-B secure BE scheme.

1) Key Stream Generation: The key stream is generated based on the key and the internal state of the generator using a function $f(K, R_i)$, where K is the key, and R_i represents the internal state at the ith step.

2) Encryption and Decryption: The ciphertext is generated by XOR ing the plaintext with the key stream $y = x \oplus z$, where x is the plaintext and z is the key stream. Decryption is done by generating the same key stream to recover the original plaintext.

III. System Model

A. Notations

For clarity, Nomenclature summarizes the main cryptographic and system notations referenced throughout the article.

B. System Overview

Our blockchain-based secure and efficient ADSB-IBBE scheme is shown in Fig. 2. At the foundational level, the blockchain network is maintained by a consortium of ground management systems, airlines, ADS-B ground stations, and airports, which act as consensus nodes. These nodes collaboratively maintain the ledger and store the identity sets S and flight plans of aerial vehicles. During flight, aerial vehicles employ our proposed ADSB-IBBE scheme to encrypt the symmetric key K, producing a ciphertext header Hdr. Subsequently, a lightweight stream cipher is applied to encrypt the message $\mathsf{AA||ME}$, ensuring that the ciphertext maintains the same length as the original message, thus preserving transmission efficiency.

In this system, the KGC generates global system parameters and securely distributes private keys to system users (i.e., aerial vehicles or ADS-B ground stations). The blockchain mainly stores user identities, providing identity sets for the vehicles before takeoff, which are used for BE. It also handles key revocation, ensuring only authorized users can decrypt messages. These interactions ensure secure and efficient communication within the ADS-B system.

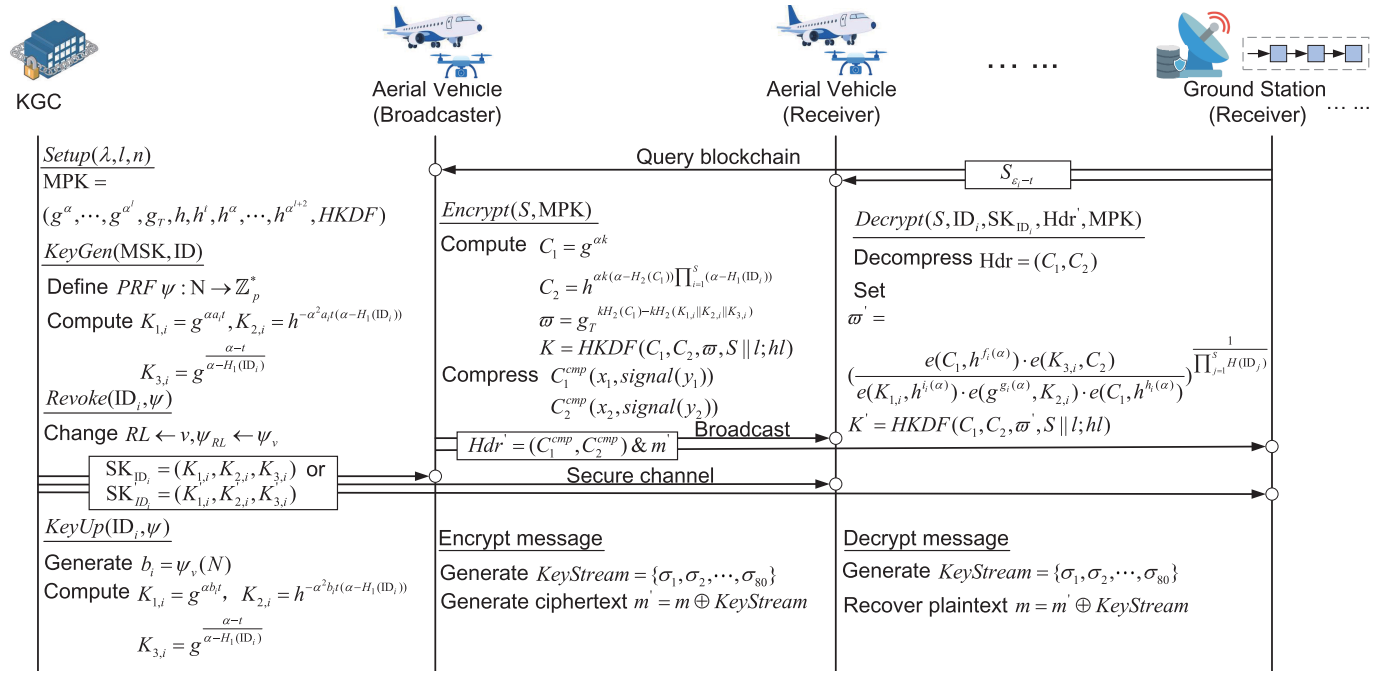

Fig. 3. Secure BE process using ADSB-IBBE and stream cipher for ADS-B systems.

C. Security Goals

1) Confidentiality: Only authorized recipients can recover the broadcast key K. Illegal users cannot recover plaintext even with ciphertext and public parameters, ensuring confidentiality over open channels. Through the IBBE algorithm, only authorized users can obtain the symmetric key and decrypt the message, preventing unauthorized access.

2) Identity Privacy Protection: Encrypted messages should not reveal the sender's and recipients' real identities, preventing aerial vehicle tracking or identity correlation attacks and protecting user privacy. Through securely storing S and using the Hdr to hide recipient identities, along with stream cipher encryption to protect the sender's identity.

3) Revocability and Updateability: The system should promptly revoke/update compromised or lost user keys and securely support key updates, ensuring revoked users cannot decrypt future encrypted content. Through blockchain-based key generation and management mechanisms, compromised keys are immediately revoked and updated, preventing key leakage and unauthorized decryption.

4) Decentralized Identity Management: Identity and revocation data should be managed via blockchain to avoid SPOF issues, ensuring the data is publicly transparent, tamper-resistant, and scalable. Through sharding blockchain, the latest valid S is used by aerial vehicles during flight, preventing identity spoofing and distributed denial of service (DDoS)-induced service disruption.

5) Message Structure Compatibility: ADS-B data fields should maintain encrypted message lengths identical to the original messages to ensure compatibility with fixed-structure, bandwidth-constrained broadcast environments. Through stream cipher keystream generation, the ciphertext adds no extra data overhead, thereby preventing transmission delays and compatibility issues.

6) Attack Resistance: The scheme must resist common attacks, including replay, forgery, user collusion, key leakage, and DDoS, ensuring robustness and security in dynamic broadcasting environments. The use of timestamps and random values prevents delay and replay attacks; dynamic generation of keystreams and hash functions prevents forgery; decentralized identity management and key revocation prevent collusion, key leakage, and DDoS attacks.

IV. Detailed Construction

In the following, we introduce our proposed ADSB-IBBE scheme, a blockchain-based identity and key management framework, and stream-cipher-based encryption mechanism. The detailed secure broadcast process incorporating ADSB-IBBE and stream encryption is illustrated in Fig. 3.

A. ADSB-IBBE Scheme

The complete ADSB-IBBE BE process for the ADS-B system is shown in Algorithm 1.

1) Setup(λ, l, n): The setup phase is primarily used to generate global parameters and publish the public key parameters to the ADS-B systems of aircraft, providing foundational support for the encryption and decryption operations of system users. This process is performed by a trusted KGC, which is responsible for establishing the system's security foundation (lines 1–7 in Algorithm 1).

1) The KGC inputs sufficiently large security parameters λ and integers n and l, where n denotes the maximum number of users in the ADS-B system, and l represents the maximum number of users that the algorithm can handle.
2) Construct the bilinear pairing group system $\mathcal{BG} = (q, p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, where the order $q \geq 2^{\lambda}$, ensuring

Algorithm 1 ADSB-IBBE BE Algorithm

———↓ ***Encryption Phase*** ↓———

Input: Security parameter λ, system parameters n, l, public parameters MPK, identity set $S = \{\mathrm{ID}_1, \ldots, \mathrm{ID}_s\}$, individual identity ID_i.

Output: Private key $\mathrm{SK}_{\mathrm{ID}_i}$, compressed ciphertext header Hdr′ and symmetric key K.

1: KGC selects λ, constructs $\mathcal{BG} = (q, p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, $q \geq 2^{\lambda}$;
2: Select $g \in_R \mathbb{G}_1, h \in_R \mathbb{G}_2$, and $\alpha \in_R \mathbb{Z}_p^*$;
3: Define $H_1 : \{0,1\}^* \to \mathbb{Z}_p^*$, $H_2 : G_1 \to \mathbb{Z}_p^*$;
4: Define $HKDF : \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T \times \{0,1\}^* \to \{0,1\}^{hl}$;
5: Initialize private key list $\mathcal{L} \leftarrow \emptyset$, revocation list $\mathcal{RL} \leftarrow \emptyset$;
6: Set $\mathsf{MSK} \leftarrow (g, \alpha)$;
7: Publish $\mathsf{MPK} = (g^{\alpha}, \ldots, g^{\alpha^l}, g_T, h, h^t, h^{\alpha}, \ldots, h^{\alpha^{l+2}}, HKDF)$.▷ /*SETUP$(\lambda, l, n)$*/

8: KGC selects $t \in_R \mathbb{Z}_p^*$;
9: Select seed $v \in_R \mathbb{Z}_p^*$, define PRF $\psi : N \to \mathbb{Z}_p^*$;
10: Generate $a_i \leftarrow \psi_v(N), i \in [1, n]$;
11: Compute $K_{1,i} = g^{\alpha a_i t}$, $K_{2,i} = h^{-\alpha^2 a_i t(\alpha - H_1(\mathsf{ID}_i))}$, $K_{3,i} = g^{(\alpha - t)/(\alpha - H_1(\mathsf{ID}_i))}$;
12: Set $\mathsf{SK}_{\mathsf{ID}_i} \leftarrow (K_{1,i}, K_{2,i}, K_{3,i})$.▷ /*KEYGEN$(\mathsf{MSK}, \mathsf{ID})$*/

13: Broadcaster selects $k \in_R \mathbb{Z}_p^*$;
14: Compute $C_1 = g^{\alpha k}$, $C_2 = h^{\alpha k(\alpha - H_2(C_1)) \prod_{i=1}^{s}(\alpha - H_1(\mathsf{ID}_i))}$;
15: Encapsulate $K \leftarrow HKDF(C_1, C_2, \varpi, S \| l; hl)$, $\varpi \leftarrow g_T^{kH_2(C_1) - kH_2(K_{1,i} \| K_{2,i} \| K_{3,i})}$;
16: Compress points $C_1^{cmp} \leftarrow (x_1, \mathsf{signal}(y_1))$, $C_2^{cmp} \leftarrow (x_2, \mathsf{signal}(y_2))$;
17: Generate compressed header $\mathsf{Hdr}' \leftarrow (C_1^{cmp}, C_2^{cmp})$.▷ /*ENCRYPT (S, MPK)*/

———↓ ***Decryption Phase*** ↓———

Input: Compressed header Hdr′, identity set $S = \{\mathrm{ID}_1, \ldots, \mathrm{ID}_s\}$, individual identity ID_i, private key $\mathrm{SK}_{\mathrm{ID}_i}$, and public parameters MPK.

Output: Symmetric key K′.

18: Receiver decompresses Hdr′ to get C_1, C_2;
19: Compute helper functions $f_i(\alpha), g_i(\alpha), h_i(\alpha), i_i(\alpha)$;
20: Compute intermediate value $\varpi' = \left(\frac{e(C_1, h^{f_i(\alpha)}) \cdot e(K_{3,i}, C_2)}{e(K_{1,i}, h^{i_i(\alpha)}) \cdot e(g^{g_i(\alpha)}, K_{2,i}) \cdot e(C_1, h^{h_i(\alpha)})} \right)^{1 / \prod_{j=1}^{s} H(\mathsf{ID}_j)}$;
21: Recover $K' \leftarrow HKDF(C_1, C_2, \varpi', S \| l; hl)$.
▷ /*DECRYPT$(S, \mathsf{ID}_i, \mathsf{SK}_{\mathsf{ID}i}, \mathsf{Hdr}', \mathsf{MPK})$*/

———↓ ***Key Update/ Revocation Phase*** ↓———

22: KGC receives key update request for user ID_i;
23: Generate new $b_i \leftarrow \psi_v(N)$;
24: Recompute $K'_{1,i} = g^{\alpha b_i t}$, $K'_{2,i} = h^{-\alpha b_i t(\alpha - H_1(\mathsf{ID}_i))}$, $K'_{3,i} = g^{\frac{\alpha - t}{\alpha - H_1(\mathsf{ID}_i)}}$;
25: Update $\mathsf{SK}'_{\mathsf{ID}_i} \leftarrow (K'_{1,i}, K'_{2,i}, K'_{3,i})$. TIKZ IMAGE

26: KGC queries $\mathcal{L}$ for ID_i;
27: Update PRF seed $RL \leftarrow v$, $\psi_{RL} \leftarrow \psi_v$;
28: Mark $\mathsf{SK}_{\mathsf{ID}_i}$ as invalid;
29: Add ID_i to $\mathcal{RL} = \{\mathsf{ID}_{RL_i}\}_{i=1}^{n}$;
30: Publish $\mathcal{RL}$ to blockchain.▷ /*KEYUP/ REVOKE(ID_i, Ψ)*/

that the system can withstand advanced cryptographic attacks. The bilinear pairing is defined as $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Randomly select generators $g \in_R \mathbb{G}_1, h \in_R \mathbb{G}_2$, and a random value $\alpha \in_R \mathbb{Z}_p^*$.

3) Define the secure Hash functions $H_1 : \{0,1\}^* \to \mathbb{Z}_p^*$ and $H_2 : \mathbb{G}_1 \to \mathbb{Z}_p^*$, where H_1 and H_2 are treated as random oracles in this context.
4) *Define the HMAC-based Key Derivation Function (HKDF):* $\mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T \times \{0,1\}^* \to \{0,1\}^{\mathrm{hl}}$, where hl represents the length of the symmetric key.
5) Initialize a private key list $\mathcal{L}$ with n users, $\mathcal{L} = \emptyset$, to store the private keys of system users. Also, initialize a revocation list $\mathcal{RL}$, $\mathcal{RL} = \emptyset$, to record the identity information of revoked users. If any illicit activities or key leaks are detected, the user's identity information is added to the revocation list.
6) Set the system main private key $\mathsf{MSK} = (g, \alpha)$ and keep it confidential. Set $e(g, h) = g_T$ and publicly release the system MPK parameters $\mathsf{MPK} = (g^{\alpha}, \ldots, g^{\alpha^l}, g_T, h, h^t, h^{\alpha}, \ldots, h^{\alpha^{l+2}}, HKDF)$.

2) KeyGen($\mathsf{MSK}, \mathsf{ID}$): The key generation phase is executed by the KGC, which inputs the identity identifiers ID_i of system users, where $i = 1, 2, \ldots, n$. The KGC performs computations using the main key MSK and the public parameters MPK, and outputs the corresponding user private key $\mathsf{SK}_{\mathsf{ID}i}$ (lines 8–12 in Algorithm 1).

1) Based on the identity identifiers ID_i of legitimate aircraft AV_i in the system, where $i \in [1, n]$, the KGC randomly selects a secret value $t \in_R \mathbb{Z}_p^*$, a random seed $v \in_R \mathbb{Z}_p^*$, and defines a pseudorandom function $\Psi : N \to \mathbb{Z}_p^*$ to generate random values $a_i = \Psi_v(N)$, where $i \in [1, n]$. The following calculations are performed:

$$\begin{aligned} K_{1,i} &= a_i t \cdot g^{\alpha} = g^{\alpha a_i t} \\ K_{2,i} &= -a_i t \alpha \cdot h^{\alpha(\alpha - H_1(\mathsf{ID}_i))} = h^{-\alpha^2 a_i t(\alpha - H_1(\mathsf{ID}_i))} \\ K_{3,i} &= \frac{1}{\alpha - H_1(\mathsf{ID}_i)} \cdot g^{\alpha - t} = g^{\frac{\alpha - t}{\alpha - H_1(\mathsf{ID}_i)}}. \end{aligned} \tag{2}$$

The user private key is set as $\mathsf{SK}_{\mathsf{ID}i} = (K_{1,i}, K_{2,i}, K_{3,i})$, which is securely transmitted to the user ID_i via a secure channel. A private key list $\mathcal{L} = \{\mathsf{ID}_i, a_i, \mathsf{SK}_{\mathsf{ID}i}\}_{i=1}^{n}$ is then formed and securely stored and maintained locally.

3) Encrypt(S, MPK): The encryption algorithm is executed by the broadcasting aerial vehicle, which inputs the identity set $S = \{\mathsf{ID}_1, \mathsf{ID}_2, \ldots, \mathsf{ID}_s\}$, where $s \leq l \leq n$, and the system's MPK. The algorithm then computes the message header Hdr and the encapsulated symmetric key K. The header Hdr is compressed into Hdr' via point compression to minimize size and reduce transmission overhead. Finally, the compressed message header Hdr' is broadcast (lines 13–17 in Algorithm 1).

1) The identity set of system users is $S = \{\mathsf{ID}_i\}_{i=1}^{s}$, where $s \leq l \leq n$. The aerial vehicle randomly selects $k \in_R \mathbb{Z}_p^*$, and computes the message header $\mathsf{Hdr} = (C_1, C_2)$ and the symmetric key K. The identity hash computation can be precomputed. The calculations are as follows:

$$C_1 = g^{\alpha k}, C_2 = h^{\alpha k(\alpha - H_2(C_1))} \prod_{i=1}^{s} (\alpha - H_1(\mathsf{ID}_i))$$

$$\varpi = g_T^{kH_2(C_1)-kH_2(K_{1,i}\|K_{2,i}\|K_{3,i})}$$
$$K = \mathrm{HKDF}(C_1, C_2, \varpi, S \parallel l; hl). \quad (3)$$

2) Point compression is applied to effectively encode C_1 and C_2, reducing the transmission data size for ME. Based on the properties of elliptic curves, the y-coordinate can be derived from the x-coordinate. For a point (x, y), given x, there are two possible values for y, namely, y and $-y \mod p$, which are modular complements of each other.
a) *Storing the x-Coordinate:* For points $C_1(x_1, y_1)$ and $C_2(x_2, y_2)$, only the x-coordinates x_1, x_2 are stored.
b) *Storing Additional Information for the y-Coordinate:* For each y-coordinate, an extra bit is stored to indicate its parity. If y is odd, the signal is stored as $\mathsf{signal}(y) = 1$; if y is even, the signal is stored as $\mathsf{signal}(y) = 0$.
For $C_1 \in \mathbb{G}_1$ and $C_2 \in \mathbb{G}_2$, after point compression, the points $C_1(x_1, y_1)$ and $C_2(x_2, y_2)$ become $C_1^{\mathrm{cmp}}(x_1, \mathsf{signal}(y_1))$ and $C_2^{\mathrm{cmp}}(x_2, \mathsf{signal}(y_2))$.
3) Finally, the aerial vehicle broadcasts the compressed message header $\mathsf{Hdr}' = (C_1^{\mathrm{cmp}}, C_2^{\mathrm{cmp}})$. Based on the system's MPK set during the algorithm's setup phase, C_2 can be computed, and therefore, both C_1^{cmp} and C_2^{cmp} can also be computed.

4) Decrypt(S, ID_i, $\mathsf{SK}_{\mathsf{ID}i}$, Hdr', MPK): The decryption phase is performed by the system user receiving the message. The compressed message header $\mathsf{Hdr}' = (C_1^{\mathrm{cmp}}, C_2^{\mathrm{cmp}})$ is parsed and decompressed to obtain the complete message header $\mathsf{Hdr} = (C_1, C_2)$. The user provides their identity identifier ID_i, private key $\mathsf{SK}_{\mathsf{ID}i}$, and the message header Hdr, and computes the symmetric key K' for the subsequent decryption of the encrypted message m (lines 18–21 in Algorithm 1).

1) The ME stores the x-coordinates of points C_1 and C_2 as well as the parity bit for the y-coordinates. These need to be decompressed to recover the original points $C_1(x_1, y_1)$ and $C_2(x_2, y_2)$, which will be used for subsequent decryption calculations. The message information C_1^{cmp}, C_2^{cmp} includes the x-values and the signal of the y-values. Using the modular square root algorithm, the possible values for y can be computed based on the x-values, and the correct y value is chosen according to the parity information.
   a) C_1^{decmp}: For C_1, we obtain two solutions for $y_{1,1} = y_1$ and $y_{1,2} = -y_1 \mod p = p - y_1$.
   If $\mathsf{signal}(y_1) = 1$, the odd solution $y_{1,1}$ is selected; if $\mathsf{signal}(y_1) = 0$, the even solution $y_{1,2}$ is selected. The decompressed point is $C_1(x_1, y_1)$.
   b) C_2^{decmp}: For C_2, we obtain two solutions for $y_{2,1} = y_2$ and $y_{2,2} = -y_2 \mod p = p - y_2$.
   If $\mathsf{signal}(y_2) = 1$, the odd solution $y_{2,1}$ is selected; if $\mathsf{signal}(y_2) = 0$, the even solution $y_{2,2}$ is selected. The decompressed point is $C_2(x_2, y_2)$.
2) The user, based on their identity identifier ID_i and private key $\mathsf{SK}_{\mathsf{ID}i}$, decrypts the message header $\mathsf{Hdr} = (C_1, C_2)$ to obtain the symmetric key K'. The hash values of the identity set and private key can be precomputed. Let

$$f_i(\alpha) = (t-\alpha)(\alpha - H_2(C_1)) \prod_{j=1, j\neq i}^{s} (\alpha - H_1(\mathsf{ID}_j)) + \frac{H_2(C_1)\prod_{j=1}^{s} H_1(\mathsf{ID}_j)}{\alpha}$$
$$g_i(r) = (\alpha - H_2(C_1)) \prod_{j=1, j\neq i}^{s} (\alpha - H_1(\mathsf{ID}_j))$$
$$h_i(\alpha) = \frac{H_2(K_{1,i} \parallel K_{2,i} \parallel K_{3,i})\prod_{j=1}^{s} H_1(\mathsf{ID}_j)}{\alpha}$$
$$i_i(\alpha) = \alpha(\alpha - H_2(C_1)) \prod_{i=1}^{s} (\alpha - H_1(\mathsf{ID}_i)). \quad (4)$$

Set

$$\varpi' = \left(\frac{e(C_1, h^{f_i(\alpha)}) \cdot e(K_{3,i}, C_2)}{e(K_{1,i}, h^{i_i(\alpha)}) \cdot e(g^{g_i(\alpha)}, K_{2,i}) \cdot e(C_1, h^{h_i(\alpha)})}\right)^{\frac{1}{\prod_{j=1}^{s} H(\mathsf{ID}_j)}}$$
$$K' = \mathrm{HKDF}(C_1, C_2, \varpi', S \parallel l; hl). \quad (5)$$

Proof: [Correctness] Assuming the identity identifier of the message receiving user $\mathsf{ID}_i \in S$, then $\varpi = \varpi'$, and hence $K = K'$. (6), as shown at the bottom of the next page. □

5) KeyUp/Revoke(ID_i, Ψ): The key update and revocation operations are performed by the KGC. When a user's private key is lost, or illegal behavior is detected, the user's key can be updated or revoked to ensure the security of the system (lines 22–30 in Algorithm 1).

1) When a user's private key is lost or compromised, the user needs to submit a key update request to the KGC, which will generate new key components. The KGC receives the user ID_i's key update request, generates a new random value $b_i = \Psi_v(N)$ using a pseudorandom function, and recalculates the values of the key components $K_{1,i}, K_{2,i}$, and $K_{3,i}$ based on the new b_i

$$K_{1,i} = g^{rb_it}, \quad K_{2,i} = h^{-rb_it(r-H_1(\mathsf{ID}_i))}$$
$$K_{3,i} = g^{\frac{r-t}{r-H_1(\mathsf{ID}_i)}}. \quad (7)$$

The KGC sends the newly generated private key $\mathsf{SK}_{\mathsf{ID}i} = (K_{1,i}, K_{2,i}$, and $K_{3,i})$ to the user ID_i via a secure communication channel, enabling the user to use the new key for subsequent operations.
2) If an aerial vehicle detects illegal activities by other vehicles or ground stations during flight, it collects the users' illegal behavior information and sends it to a legitimate ground station, or if a legitimate ground station detects and collects the illegal behavior of a user, it sends the information to the KGC. After KGC verifies the information, it will perform the key revocation operation to prevent the user from continuing to decrypt the system's messages. If a user's key needs to be permanently revoked (e.g., if KGC detects illegal behavior independently), the revocation operation will also be executed. The KGC queries the user's private

key list $\mathcal{L}$ based on the user's ID_i, changes the pseudorandom function Ψ_v seed to $RL \leftarrow v, \Psi_{RL} \leftarrow \Psi_v$, thus altering the user's private key $\mathsf{SK}_{\mathsf{ID}i}$. The illegal user will no longer be able to decrypt new message data. Subsequently, the KGC adds the illegal user's identity to the revocation list $\mathcal{RL} = \{\mathsf{ID}_{RL_i}\}_{i=1}^{n}$. If the illegal users believe the revocation is unreasonable, they can file an appeal with the KGC. If the appeal is approved, the KGC will regenerate a new key for the users and send it to them via a secure channel. The KGC publishes an announcement on the blockchain containing the revocation list $\mathcal{RL} = \{\mathsf{ID}_{RL_i}\}_{i=1}^{n}$, ensuring that other aerial vehicles in the system can exclude the revoked user identity before flight, preventing the illegal user from accessing broadcast-encrypted messages.

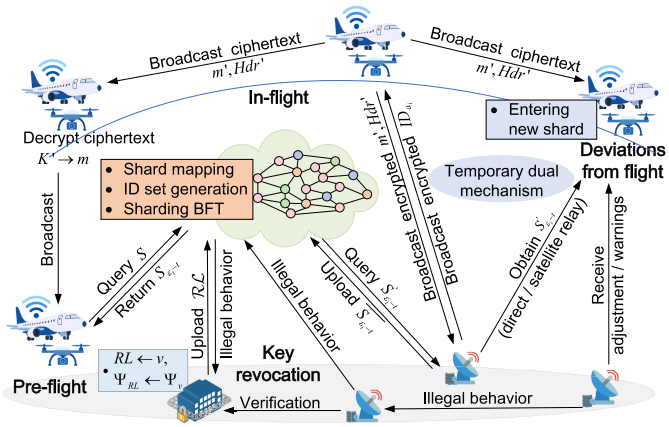

Fig. 4. Blockchain-based user interaction workflow for secure ADS-B operations.

B. Sharding Blockchain Design

The ADSB-IBBE scheme adopts a consortium-based sharding blockchain to manage the identity information of aerial vehicles, ground stations, and flight plans. Ground management systems, airlines, ADS-B ground stations, and airports serve as authorized blockchain nodes, collaboratively maintaining the ledger within each shard. We assume that the underlying blockchain infrastructure, including shard consensus and intershard communication, is secure and functions correctly. Potential attacks targeting the base blockchain protocol, such as shard takeover or consensus manipulation, are out of the scope of this article.

Each block in the sharding blockchain contains metadata for its corresponding spatial–temporal shard. The block header records the shard identifier, timestamp, previous block hash, and BFT consensus signature, while the block body stores identity sets, flight plan hashes, and revocation records, if any. Before takeoff, each aerial vehicle retrieves the relevant identity sets based on its planned flight path and precomputes encryption parameters. This ensures secure in-flight BE and decryption, and maintains system security by updating identity sets if the flight path deviates. The key revocation mechanism is activated upon detection of illegal behavior, ensuring security. The user interaction process based on the blockchain system is depicted in Fig. 4, enhancing overall security, flexibility, and load balancing.

Moreover, the consortium-based sharding blockchain enhances scalability through shard-level parallelism. Each shard independently manages its identity set and performs BE only within its scope, reducing the global overhead of managing all users. The sharding BFT consensus ensures fast, lightweight agreement, while cross-shard synchronization facilitates efficient identity updates across multiple shards when necessary, such as when a flight path deviates and enters a new spatial–temporal shard. These properties confirm that the blockchain layer is scalable, efficient, and well-suited for real-time ADS-B operations.

1) Preflight Process: Before takeoff, the sharding blockchain system initializes data based on the flight plan, including shard mapping, ID set generation, parameter calculation, and identity information retrieval.

1) *Shard Mapping:* The flight plan data, including the departure point, route, flight time, and target area, is submitted to the sharding blockchain system by the ground management system or airline. The system partitions the airspace into fixed-size geographical units,

$$
\begin{aligned}
\omega_1 &= e(C_1, h^{f_i(\alpha)}) \cdot e(K_{3,i}, C_2) \\
&= e\left(g^{\alpha k}, h^{(t-\alpha)(\alpha-H_2(C_1))\prod_{j=1,j\neq i}^{s}(\alpha-H_1(\mathsf{ID}_j))+\frac{H_2(C_1)\prod_{j=1}^{s}H_1(\mathsf{ID}_j)}{\alpha}}\right) \cdot e\left(g^{\frac{\alpha-t}{\alpha-H_1(\mathsf{ID}_j)}}, h^{\alpha k(\alpha-H_2(C_1))\prod_{j=1}^{s}(\alpha-H_1(\mathsf{ID}_j))}\right) \\
&= e(g,h)^{\alpha k(t-\alpha)(\alpha-H_2(C_1))\prod_{j=1,j\neq i}^{s}(\alpha-H_1(\mathsf{ID}_j))+kH_2(C_1)\prod_{j=1}^{s}H_1(\mathsf{ID}_j)} \cdot e(g,h)^{\alpha k(\alpha-t)(\alpha-H_2(C_1))\prod_{j=1,j\neq i}^{s}(\alpha-H_1(\mathsf{ID}_j))} \\
&= e(g,h)^{kH_2(C_1)\prod_{j=1}^{s}H_1(\mathsf{ID}_j)} \\
\omega_2 &= e(K_{1,i}, h^{i_i(\alpha)}) \cdot e(g^{g_i(\alpha)}, K_{2,i}) \cdot e(C_1, h^{h_i(\alpha)}) \\
&= e\left(g^{\alpha a_i t}, h^{\alpha(\alpha-H_2(C_1))\prod_{i=1}^{s}(\alpha-H_1(\mathsf{ID}_i))}\right) \cdot e\left(g^{(\alpha-H_2(C_1))\prod_{j=1,j\neq i}^{s}(\alpha-H_1(\mathsf{ID}_j))}, h^{-\alpha^2 a_i t(\alpha-H_1(\mathsf{ID}_i))}\right) \\
&\cdot e\left(g^{\alpha k}, h^{\frac{H_2(K_{1,i}\|K_{2,i}\|K_{3,i})\prod_{j=1}^{s}H_1(\mathsf{ID}_j)}{\alpha}}\right) = e(g,h)^{kH_2(K_{1,i}\|K_{2,i}\|K_{3,i})\prod_{j=1}^{s}H_1(\mathsf{ID}_j)} \\
\varpi' &= \left(\frac{e(C_1, h^{f_i(\alpha)}) \cdot e(K_{3,i}, C_2)}{e(K_{1,i}, C_2) \cdot e(C_1, g_i(\alpha)K_{2,i} \cdot h^{h_i(\alpha)})}\right)^{\frac{1}{\prod_{j=1}^{s}H(\mathsf{ID}_j)}} = \left(\frac{\omega_1}{\omega_2}\right)^{\frac{1}{\prod_{j=1}^{s}H(\mathsf{ID}_j)}} = g_T^{kH_2(C_1)-kH_2(K_{1,i}\|K_{2,i}\|K_{3,i})} = \varpi \\
K &= K'
\end{aligned}
\tag{6}
$$

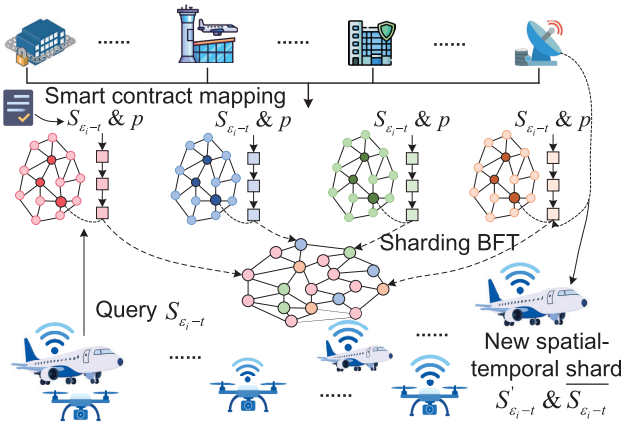

Fig. 5. Spatial–temporal shard identity set data storage and query on a sharding blockchain.

each associated with a unique shard identifier ε_i that manages the identity set S_{ε_i} of all aerial vehicles and ADS-B ground stations in that region. The time axis is then divided into fixed windows t (e.g., daily), which are combined with the spatial partitions to form spatial–temporal shards $\varepsilon_i - t$. In this design, each flight path is mapped to its corresponding set of spatial–temporal shards, enabling the aerial vehicle to maintain accurate identity association and orderly key management throughout the flight. Blockchain nodes run BFT consensus protocols and execute smart contracts to map flight plan data, ensuring accurate identity mapping to the relevant shard.

2) *ID Set Generation:* Each shard corresponds to a geographic area, where blockchain nodes store the identity set S_{ε_i-t} and flight plan information p for all aerial vehicles and ADS-B ground stations within that area over a specific time period, and automatically share relevant data with other shard nodes via smart contracts to ensure data consistency and real-time availability, as shown in Fig. 5. The sharding BFT consensus mechanism ensures secure sharing of data across shards through signing and voting mechanisms. Even if some nodes experience attacks or failures, the system remains fault-tolerant, ensuring data consistency and validity. When an aerial vehicle's flight path crosses multiple shards, the system calculates the required spatial–temporal shard set and generates the corresponding identity sets S_{ε_i-t} in advance. This design does not rely on real-time synchronization during flight and is thus unaffected by propagation latency, ensuring secure communication throughout the subsequent flight.

3) *Prefetching Spatial–Temporal Shard Identity Information:* Before takeoff, the aerial vehicle queries the blockchain to retrieve the identity sets S_{ε_i-t} for all spatial–temporal shards it will enter based on its flight plan, ensuring it can perform the necessary BE and decryption during flight. At the same time, the aerial vehicle precalculates the related parameters within $f_i(r)$, $g_i(r)$, $h_i(r)$, and $i_i(r)$ for broadcast decryption in these shards. These parameters are stored locally to avoid delays or excessive resource usage during flight. The identity set S_{ε_i-t} received before takeoff remains valid as long as the aerial vehicle stays on course. These prefetched data are stored temporarily on the aerial vehicle and can be discarded after the flight, resulting in minimal memory consumption and no risk of overloading the system.

2) In-Flight Process: During flight operations, due to network limitations, communication latency, bandwidth constraints, and aerial vehicles' task independence, real-time blockchain queries are avoided. Instead, aerial vehicles utilize preloaded spatial–temporal shard identity sets and precalculated parameters acquired before takeoff, ensuring efficient and secure ADS-B BE and decryption.

1) *BE:* Aerial vehicles locally query the identity set S_{ε_i-t} corresponding to the spatial–temporal shard and utilize public parameters of the ADSB-IBBE algorithm to generate a symmetric key K. The plaintext message m of AA and ME fields in the ME is encrypted using a stream cipher, producing ciphertext m' of identical length to the original plaintext. The ciphertext m' occupies the original fixed positions in the AA and ME fields, while the compressed message header Hdr′ is placed in the ME and PI fields of subsequent ME. Consequently, only aerial vehicles and ground stations within the same spatial–temporal shard can decrypt the broadcast message, effectively preventing illegal entities from intercepting or altering message content.

2) *Decryption Operation:* When an aerial vehicle or ground station within the same spatial–temporal shard receives an ADS-B broadcast message, it first extracts ciphertext m' from the AA and ME fields and the compressed message header Hdr′. After decompressing Hdr′, the receiver obtains the message header $\text{Hdr} = (C_1, C_2)$. Using locally precalculated parameters and the identity set S_{ε_i-t}, the receiver performs the broadcast decryption to recover the symmetric key $K' = K$. Finally, the receiver uses the K' to decrypt ciphertext m' from the AA and ME fields via the stream cipher, recovering the plaintext m for subsequent message processing and analysis.

3) Aerial Vehicle Deviations From Flight Path: During flight, aerial vehicles may deviate from their planned path due to adverse weather, navigation errors, or unforeseen circumstances. Two scenarios arise: remaining in the same shard or entering a new spatial–temporal shard. The system employs mechanisms to ensure continuity in BE and decryption.

1) *Remaining Within the Same Spatial–Temporal Shard:* If the aerial vehicle a_i deviates from its planned flight path but does not enter a new spatial–temporal shard, it continues to operate within the current shard. In this case, the aerial vehicle a_i continues to use the identity set S_{ε_i-t} and precalculated parameters stored locally for BE and decryption of ME. The aerial vehicle a_i may receive trajectory adjustment suggestions and real-time warnings from legitimate ground stations or nearby aerial vehicles. These instructions help the vehicle correct its deviation,

ensuring it stays on the intended path while maintaining the original encryption mechanism.

2) *Entering a New Spatial–Temporal Shard:* If the aerial vehicle a_i deviates from its path and enters a new spatial–temporal shard, it can relay through a satellite-based ADS-B system to a legitimate ground station, or communicate with the ground station directly, for blockchain-based queries to obtain the identity set S'_{ε_i-t} of the new shard. The aerial vehicle a_i updates its identity ID_{a_i} into the identity set S'_{ε_i-t}, resulting in an incremental spatial–temporal shard identity set $\overline{S_{\varepsilon_i-t}}$. Legitimate ground stations then upload the updated identity set $\overline{S_{\varepsilon_i-t}}$ to the blockchain. Nearby ground stations broadcast the encrypted ID_{a_i} to surrounding aerial vehicles. Once decrypted by nearby vehicles, these vehicles add ID_{a_i} to their local spatial–temporal shard identity set S'_{ε_i-t}, resulting in the same incremental identity set $\overline{S_{\varepsilon_i-t}}$. Both the aerial vehicle a_i and nearby vehicles or ground stations can then perform BE and decryption within the updated spatial–temporal shard, creating a temporary dual encryption mechanism alongside the original identity set encryption. Aerial vehicles that have not updated their identity sets can still use the original identity set S'_{ε_i-t} for encryption.

At the same time, the aerial vehicle may continue to receive warnings or trajectory adjustment suggestions from legitimate ground stations or other aerial vehicles. These instructions help the vehicle correct its path and return to the planned route, thus restoring the original BE mechanism.

4) Key Revocation: When an aerial vehicle or ground station detects illegal behavior from a legitimate user, key revocation must be quickly executed to prevent the illegal user from continuing to decrypt broadcast messages. Blockchain technology ensures a transparent, decentralized, and traceable revocation mechanism.

1) *Illegal Behavior Detection and Blockchain Upload:* When a user detects illegal behavior from another, it collects relevant data, such as illegal identity modification or message forgery, using predefined identifiers. If the user is illegal, legitimate users avoid interaction. If the user is a legitimate aerial vehicle with suspicious behavior, the illegal behavior information is broadcast-encrypted and sent to a legitimate ground station, or alternatively relayed via satellite-based ADS-B for quick transmission. The legitimate ground station decrypts the information and either uploads it to the blockchain for BFT consensus within the shard system or directly forwards it to the KGC for verification. The KGC then confirms the behavior and initiates key revocation.
2) *Key Revocation Operation:* The KGC retrieves the illegal user's identity ID from the blockchain and searches the private key list $\mathcal{L} = \{\mathsf{ID}_i, a_i, \mathsf{SK}_{\mathsf{ID}_i}\}_{i=1}^{n}$ to extract the private key $\mathsf{SK}_{ID_i} = (K_{1,i}, K_{2,i}, K_{3,i})$. If the behavior is confirmed as illegal, the KGC revokes the key by updating the seed of the pseudorandom function $\Psi_v(N)$ with $RL \leftarrow v, \Psi_{RL} \leftarrow \Psi_v$, preventing the user from decrypting new messages. The KGC then adds the revoked user's identity ID to the revocation list $\mathcal{RL} = \{\mathsf{ID}_{RL_i}\}_{i=1}^{n}$ and publishes it to the blockchain. All blockchain nodes validate the list, reach BFT consensus, and update the spatial–temporal shard identity sets. Subsequent aerial vehicles will obtain the updated shard identity set before takeoff, excluding revoked users, ensuring they cannot decrypt messages. A revoked user can appeal to the KGC for a review; if successful, a new key will be issued, and the user will be removed from the revocation list.

C. Stream Cipher Algorithm

In ME BE, a stream cipher algorithm is employed to efficiently encrypt the AA and ME fields (80 bits in total) of the message plaintext, ensuring data transmission security. This generates an 80-bit ciphertext without expanding the message length or modifying the binary structure. It can be seamlessly embedded into the ME, maintaining full compliance with the 1090ES standard. The implementation and processing flow of the stream cipher algorithm are described for both the sender and receiver as follows, as shown in Algorithm 2.

1) Sender Processing Flow: The sender is responsible for the entire encryption process, including IV generation, keystream generation, and message encryption.

1) *Timestamp Generation:* The aerial vehicle, as the sender, synchronizes its transmitting device to GPS time using GNSS, ensuring the transmission timestamp is precise to the hour (line 2 in Algorithm 2)

$$\mathrm{ts}_{\mathsf{send}} = \mathsf{truncate}(\mathrm{GNSS\_Time}, \mathsf{hour}). \tag{8}$$

2) *IV Generation:* The sender generates the IV using the timestamp $\mathrm{ts}_{\mathsf{send}}$, the spatial–temporal shard identity set S_{ε_i-t}, and the message compressed header Hdr'. The IV is computed using the lightweight Blake2s hash algorithm, with the output length set to 80 bits (line 3 in Algorithm 2)

$$\mathrm{IV}_{\mathsf{send}} = \mathrm{Blake2}_{80}\begin{pmatrix} \mathrm{hash}(\mathrm{ts}_{\mathsf{send}}) \,\|\, \mathrm{hash}(\mathrm{Hdr'}) \,\| \\ \mathrm{hash}\,(\mathrm{ID}_1 \oplus \cdots \oplus \mathrm{ID}_s) \end{pmatrix}. \tag{9}$$

3) *Key Stream Generation:* To achieve efficient BE of ME, the plaintext consists of a 24-bit AA field representing the aerial vehicle's identity ID_i and a 56-bit ME field representing the message data m_i. We reference the key stream design principles of the Trivium algorithm [42]. Trivium has been extensively analyzed and remains secure against known stream cipher attacks, including guess-and-determine and algebraic attacks, with no practical attack successfully breaking it. The aerial vehicle uses a symmetric key K and a dynamically generated IV to produce an 80-bit key stream, $\mathsf{KeyStream}$, for encrypting the message.
In the initialization phase, the key K and the IV are loaded into a 288-bit state register, which consists of three independent registers, as shown in Fig. 6 (lines 4–13 in Algorithm 2).

Algorithm 2 ADS-B Stream Cipher Algorithm

———↓ ***Sender Processing Flow*** ↓———

Input: Plaintext message $m \leftarrow$ AA||ME, symmetric key K, timestamp ts $_{\text{send}}$, spatial-temporal shard identity set $S = \{\text{ID}_1, \text{ID}_2, \ldots, \text{ID}_s\}$, compressed header Hdr′.

Output: Key stream KeyStream, Ciphertext message m'.

1: Obtain $m \leftarrow$ AA || ME; # *Message retrieved by aerial vehicle (Sender)*
2: Generate $ts_{\text{send}} \leftarrow \text{truncate}(GNSS\_Time, \text{hour})$;
3: Generate $IV_{\text{send}} \leftarrow \text{Blake2}_{80}\left(\begin{smallmatrix} hash(t_s \text{ send }) \| \text{ hash }(\text{Hdr}') \\ \| \text{ hash } (\text{ID}_1 \oplus \cdots \oplus \text{ID}_s) \end{smallmatrix}\right)$;
4: $\text{A}\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\} \leftarrow \{K_1, K_2, \ldots, K_{80}, 0, \ldots, 0\}$, $\text{B}\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\} \leftarrow \{IV_1, IV_2, \ldots, IV_{80}, 0, 0, 0, 0\}$, $\text{C}\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\} \leftarrow \{0, 0, \ldots, 0, 1, 1, 1\}$; ①*Registers initialization phase*
5: **for** $i = 1$ to N **do**
6: $\mu_1 \leftarrow \gamma_{66} + \gamma_{93}$, $\mu_2 \leftarrow \gamma_{162} + \gamma_{177}$, $\mu_3 \leftarrow \gamma_{243} + \gamma_{288}$;
7: $A \leftarrow \{\mu_3, s_1, \ldots, s_{92}\}, B \leftarrow \{\mu_1, s_{94}, \ldots, s_{176}\}, C \leftarrow \{\mu_2, s_{178}, \ldots, s_{287}\}$. ②*Warm-up phase*
8: Generate $\text{KeyStream} = \{\sigma_1, \sigma_2, \ldots, \sigma_{80}\}$:
9: **for** $i = 1$ to 80 **do**
10: $\mu_1 \leftarrow \gamma_{66} + \gamma_{93}$, $\mu_2 \leftarrow \gamma_{162} + \gamma_{177}$, $\mu_3 \leftarrow \gamma_{243} + \gamma_{288}$;
11: $\sigma_i \leftarrow \mu_1 + \mu_2 + \mu_3$;
12: $\mu_1 \leftarrow \mu_1 + \gamma_{91} \cdot \gamma_{92} + \gamma_{171}$, $\mu_2 \leftarrow \mu_2 + \gamma_{175} \cdot \gamma_{176} + \gamma_{264}$, $\mu_3 \leftarrow \mu_3 + \gamma_{286} \cdot \gamma_{287} + \gamma_{69}$;
13: $A \leftarrow \{\mu_3, \gamma_1, \ldots, \gamma_{92}\}$, $B \leftarrow \{\mu_1, \gamma_{94}, \ldots, \gamma_{176}\}$, $C \leftarrow \{\mu_2, \gamma_{178}, \ldots, \gamma_{287}\}$. ③*Key stream generation phase*
14: Encrypt the message $m' \leftarrow m \oplus \text{KeyStream}$. ▷ /*ENCRYPT*/

———↓ ***Receiver Processing Flow*** ↓———

Input: Ciphertext message $m' \leftarrow$ AA|| ME, Symmetric key K, Timestamp t_{srecv} , Spatial-temporal shard identity set $S = \{\text{ID}_1, \text{ID}_2, \ldots, \text{ID}_s\}$, Compressed header Hdr′.

Output: Key stream KeyStream, Plaintext message $m \leftarrow$ AA||ME.

15: Obtain $m' \leftarrow$ AA || ME, Hdr′; # *Message received by aerial vehicle or ground station (Receiver)*
16: Record $ts_{\text{send}} \leftarrow \text{truncate}(GNSS\_Time, \text{hour})$;
17: Generate $ts_{\text{candi}} \leftarrow \{ts_{\text{recv}}, ts_{\text{recv}} \pm 1\text{h}\}$;
18: **for** $ts \in ts_{candi}$ **do**
19: $IV_{\text{candi}} \leftarrow \text{Blake}\ 2_{80}\left(\begin{smallmatrix} \text{hash}(ts) \| \text{ hash } (\text{Hdr}') \\ \| \text{hash}(\text{ID}_1 \oplus \cdots \oplus \text{ID}_s) \end{smallmatrix}\right)$.. # *Compute candidate IVs*
20: $\text{A}\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\} \leftarrow \{K_1, K_2, \ldots, K_{80}, 0, \ldots, 0\}$, $\text{B}\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\} \leftarrow \{IV_1, IV_2, \ldots, IV_{80}, 0, 0, 0, 0\}$, $\text{C}\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\} \leftarrow \{0, 0, \ldots, 0, 1, 1, 1\}$; ①*Registers initialization phase*
21: Perform N shifts and feedbacks (same as encryption step); ② *Warm-up phase*
22: Generate KeyStream (same as encryption step); ③ *Key stream generation phase*
23: Decrypt the message $m \leftarrow m' \oplus \text{KeyStream}$.▷ /*DECRYPT*/

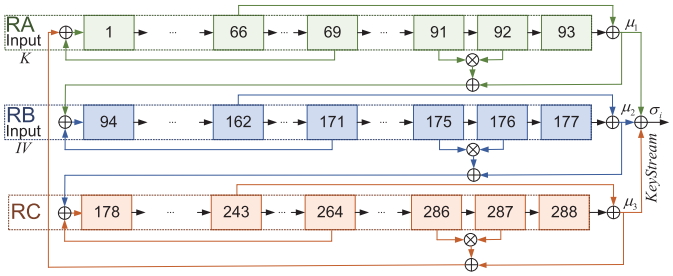

Fig. 6. Key stream generation mechanism.

Register $\text{A}\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\}$*:* Load the key K, where the first 80 bits represent the actual key values $\{K_1, K_2, \ldots, K_{80}\}$, and the remaining 13 bits are set to 0

$$\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\} \leftarrow \{K_1, K_2, \ldots, K_{80}, 0, \ldots, 0\}. \quad (10)$$

Register $\text{B}\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\}$*:* Load the IV, where the first 80 bits represent the actual vector values $\{\text{IV}_1, \text{IV}_2, \ldots, \text{IV}_{80}\}$, and the remaining 4 bits are set to 0

$$\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\} \leftarrow \{\text{IV}_1, \text{IV}_2, \ldots, \text{IV}_{80}, 0, 0, 0, 0\}. \quad (11)$$

Register $\text{C}\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\}$*:* Initialize as

$$\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\} \leftarrow \{0, \ldots, 0, 1, 1, 1\}. \quad (12)$$

After loading, the registers enter the warm-up phase, where several shift and feedback operations are performed to ensure complete initialization and provide a higher degree of randomness for the key stream generation.

At each iteration, three feedback variables μ_1, μ_2, μ_3 are computed

$$\mu_1 = \gamma_{66} + \gamma_{93}, \mu_2 = \gamma_{162} + \gamma_{177}, \mu_3 = \gamma_{243} + \gamma_{288}. \quad (13)$$

The register states are updated based on the feedback variables

$$\begin{aligned} &\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\} \leftarrow \{\mu_3, s_1, \ldots, s_{92}\} \\ &\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\} \leftarrow \{\mu_1, s_{94}, \ldots, s_{176}\} \\ &\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\} \leftarrow \{\mu_2, s_{178}, \ldots, s_{287}\}. \end{aligned} \quad (14)$$

After the warm-up phase, the registers enter the key stream generation phase, producing an 80-bit key stream $\text{KeyStream} = \{\sigma_1, \sigma_2, \ldots, \sigma_{80}\}$. Each key stream bit σ_i is generated by computing the feedback variables μ_1, μ_2, and μ_3

$$\mu_1 = \gamma_{66} + \gamma_{93}, \mu_2 = \gamma_{162} + \gamma_{177}, \mu_3 = \gamma_{243} + \gamma_{288}. \quad (15)$$

The current key stream bit is calculated as

$$\sigma_i = \mu_1 + \mu_2 + \mu_3. \quad (16)$$

The register states are then updated based on the nonlinear feedback formula

$$\begin{aligned} \mu_1 &= \mu_1 + \gamma_{91} \cdot \gamma_{92} + \gamma_{171} \\ \mu_2 &= \mu_2 + \gamma_{175} \cdot \gamma_{176} + \gamma_{264} \\ \mu_3 &= \mu_3 + \gamma_{286} \cdot \gamma_{287} + \gamma_{69}. \end{aligned} \quad (17)$$

The register states are shifted to the right, and the results of μ_1, μ_2, and μ_3 are injected into the first position of the corresponding registers

$$\begin{aligned} &\{\gamma_1, \gamma_2, \ldots, \gamma_{93}\} \leftarrow \{\mu_3, \gamma_1, \ldots, \gamma_{92}\} \\ &\{\gamma_{94}, \gamma_{95}, \ldots, \gamma_{177}\} \leftarrow \{\mu_1, \gamma_{94}, \ldots, \gamma_{176}\} \end{aligned}$$

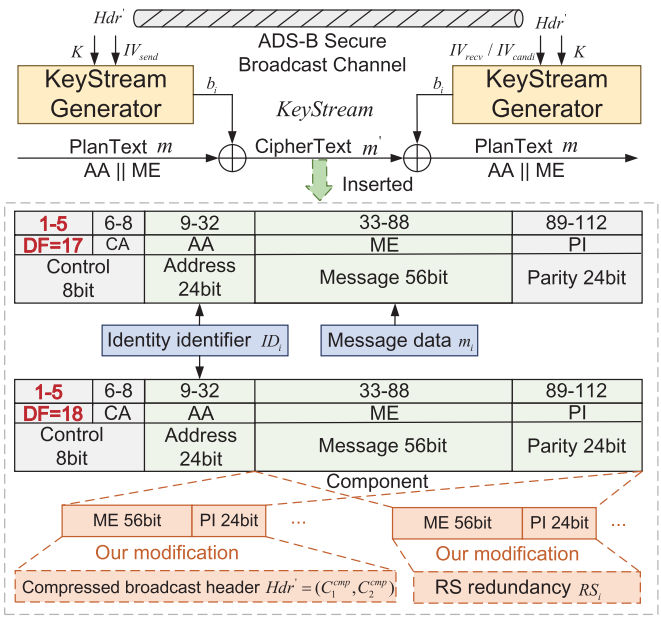

Fig. 7. Stream cipher encryption and decryption of ME and insertion of message ciphertext.

$$\{\gamma_{178}, \gamma_{179}, \ldots, \gamma_{288}\} \leftarrow \{\mu_2, \gamma_{178}, \ldots, \gamma_{287}\}. \tag{18}$$

The feedback variables μ_1, μ_2, and μ_3 are recomputed, and the process is repeated for 80 iterations, generating the 80-bit key stream $\mathsf{KeyStream} = \{\sigma_1, \sigma_2, \ldots, \sigma_{80}\}$, which is used to encrypt the 80-bit ME plaintext.

4) *Message Encryption:* The encryption targets the 24-bit AA field and 56-bit ME field of the ME, forming the message plaintext m (line 14 in Algorithm 2)

$$m = \text{AA} \parallel \text{ME}. \tag{19}$$

The plaintext is XOR ed with the generated key stream to produce the ciphertext m'

$$m' = m \oplus \mathsf{KeyStream}. \tag{20}$$

The ciphertext m' is embedded into the AA and ME fields of the ME. The BE compressed header $\mathsf{Hdr}' = (C_1^{\text{cmp}}, C_2^{\text{cmp}})$ is inserted into the subsequent message's ME and PI fields. The AA field now stores the unique ciphertext AA, enabling the receiver to identify which aerial vehicle the Hdr' belongs to. To reduce message space usage, Hdr' can be broadcast periodically, either at fixed time intervals (e.g., every second) or after a set number of messages (e.g., every 10 messages), rather than being appended to each message. This minimizes the overhead of Hdr' and increases the transmission frequency of message data, as shown in Fig. 7.

2) Receiver Processing Flow: The aerial vehicle or ground station, as the receiver, is responsible for decrypting the received ADS-B broadcast messages. The core process includes timestamp generation, IV generation, and key stream generation.

1) *Timestamp Generation:* The receiver also synchronizes with GNSS time and records the message reception timestamp truncated to the hour precision ts_{recv} (lines 16 and 17 in Algorithm 2)

$$\text{ts}_{\text{recv}} = \mathsf{truncate}(\text{GNSS\_time}, \mathsf{hour}). \tag{21}$$

If the message ciphertext spans across an hour boundary, the receiver can try the timestamps from the previous, current, and next hour to generate candidate timestamps ts_{candi}, ensuring correct decryption during the hour switch, with tolerance for clock drift and transmission delays within ± 1 h

$$\text{ts}_{\text{candi}} = \{\text{ts}_{\text{recv}}, \text{ts}_{\text{recv}} \pm 1\text{h}\}. \tag{22}$$

2) *IV Generation:* In most cases, the sender and receiver will use the same timestamp within the same hour. Therefore, the receiver calculates the IV using the current hour-level timestamp ts_{recv}. The IV is computed based on the received timestamp, the spatial–temporal shard identity set, and the Hdr' (lines 18 and 19 in Algorithm 2)

$$\text{IV}_{\text{recv}} = \text{Blake 2 s}_{80}\left(\begin{array}{c}\text{hash}\ (\text{ts}_{\text{recv}}) \parallel \text{hash}\ (\text{Hdr}') \\ \parallel \text{hash}\ (\text{ID}_1 \oplus \cdots \oplus \text{ID}_s)\end{array}\right). \tag{23}$$

If the message crosses an hour boundary, the receiver may additionally use the previous and next hour timestamps $\text{ts}_{\text{recv}} \pm 1\text{h}$, computing three candidate IVs in total

$$IV_{\text{candi1}} = \text{Blake 2}s_{80}\left(\begin{array}{c}\text{hash}\ (t_{\text{recv}}) \parallel \text{hash}\ (\text{Hdr}') \\ \parallel \text{hash}\ (\text{ID}_1 \oplus \cdots \oplus \text{ID}_s)\end{array}\right) \tag{24}$$

$$I_{\text{candi2}} = \text{Blake 2 s}_{80}\left(\begin{array}{c}\text{hash}\ (\text{ts}_{\text{recv}} - 1\ \text{h}) \parallel \text{hash}\ (\text{Hdr}') \\ \parallel \text{hash}\ (\text{ID}_1 \oplus \cdots \oplus \text{ID}_s)\end{array}\right). \tag{25}$$

The receiver then verifies which IV is correct by checking if the decrypted message matches the expected message format.

3) *Key Stream Generation:* Based on the symmetric key K (recovered from the broadcast decryption) and the generated IV_{recv} or one of the candidate IVs, the receiver generates the same key stream $\mathsf{KeyStream}$ using the stream cipher (lines 20–22 in Algorithm 2).

4) *Message Decryption:* The aerial vehicle or ground station first checks the message's PI field for transmission errors. Since message errors are rare, any errors encountered are discarded, and the message is not processed further (line 23 in Algorithm 2).

After confirming message integrity, the receiver decrypts the message by XOR ing the ciphertext with the key stream, recovering the plaintext AA and ME fields, as shown in Fig. 7

$$m = m' \oplus \mathsf{KeyStream} = \text{AA} \parallel \text{ME}. \tag{26}$$

For messages that cross the hour boundary, the receiver checks the decrypted plaintexts generated from both $\text{IV}_{\text{candi1}}$ and $\text{IV}_{\text{candi2}}$, verifying which IV correctly decrypts the message. This issue has minimal impact and can sometimes be ignored.

3) Improve Message Transmission Reliability: We use Reed–Solomon (RS) coding to recover lost messages in real-world environments, typically with a packet loss rate of around 33% [41]. RS(12 373) can recover messages with around 40% packet loss, ensuring full decryption and restoration. The message ciphertext and broadcast compressed header Hdr' consist of 8 messages, with five additional RS redundant messages RS_i, requiring a total of 13 messages per group. With a broadcast rate of 6.2 messages/s [43], a group is transmitted in about 2 s, which is acceptable with the current system performance. Increasing the broadcast rate will reduce transmission time and improve system efficiency and real-time performance [6].

V. Performance Analysis

In this study, we compare the proposed ADSB-IBBE algorithm with conventional SE methods for ADS-B systems [13], [14] and several classical BE schemes [15], [16], [17], [18], [19], [20], evaluating their computational and communication costs. The experiments aim to demonstrate the advantages of our stream cipher-based ADSB-IBBE in terms of computational efficiency and message overhead. This study focuses on algorithm-level performance evaluation, rather than full-system deployment, aiming to isolate and fairly compare the computational and communication overhead of various encryption schemes under a consistent environment. All tests were conducted on a server running CentOS Linux 7 with sufficient processing power and memory to ensure the accuracy and reliability of the results. Note that the experiments are designed to evaluate the computational and communication performance of the proposed ADSB-IBBE scheme. The security properties of the scheme are formally proven in the Appendix, following standard cryptographic practice, rather than through empirical attack simulations.

We employed the BN128 curve from the pairing-based cryptography (PBC) library as the underlying elliptic curve, due to its 128-bit security level and suitability for resource-constrained devices. The BN128 (alt_bn128) curve uses a 254-bit prime field and supports efficient bilinear pairing operations, making it well-suited for pairing-based encryption schemes. Its computational performance is particularly advantageous for low-power and low-bandwidth environments, effectively reducing both computation and storage overhead.

A. Computational Cost

The evaluation involves various cryptographic operations, including $T_{\text{bp}}, T_{\text{et}}, T_{\text{sm}_1}, T_{\text{sm}_2}, T_{\text{pa}_1}, T_{\text{pa}_2}, T_{\text{paT}}, T_{\text{hash}}, T_{\oplus}, T_{\text{aes}}$, and T_{stream}, with detailed definitions provided in the Nomenclature. To reduce measurement noise and improve the reliability of the results, each operation was executed NUM_ITERATIONS = 6000 times, and the average runtime (in milliseconds) was recorded. This setup enables accurate evaluation of the computational performance of ADSB-IBBE compared to other encryption schemes under different cryptographic tasks.

We comprehensively evaluate the computational cost of the proposed ADSB-IBBE scheme against representative encryption schemes [13], [14], [15], [16], [17], [18], [19], [20] to demonstrate its efficiency advantage. First, we analyze the overhead involved in encrypting and decrypting the symmetric key K and encrypted broadcast header Hdr, which reflects the computational resources required for key generation and recovery. Table II summarizes the computational cost of each scheme in both encryption and decryption phases for K/Hdr. In addition, we examine the cost of key update/revocation operations, as well as the complexity of public/private key computations, to evaluate the flexibility of key management across schemes. Subsequently, we assess the overhead associated with encrypting and decrypting the message data m. Table III presents the performance comparison in terms of computational cost for message-level encryption and decryption, highlighting each scheme's efficiency in broadcast data transmission scenarios. Finally, a comprehensive comparison is conducted to demonstrate the overall performance advantage of the proposed ADSB-IBBE scheme.

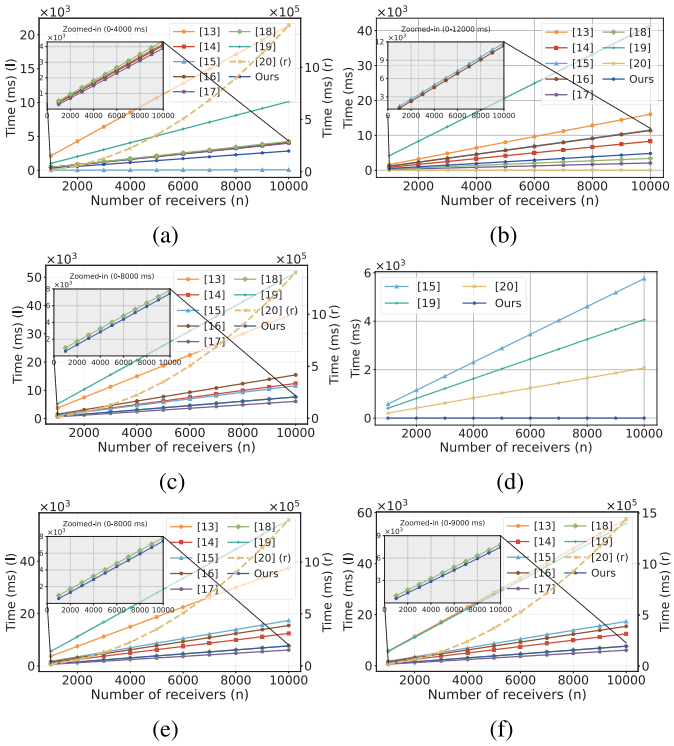

Fig. 8. Comparison of computation costs across different schemes for encryption, decryption, update/revocation, and message processing. (a) Encryption computation cost (Hdr/K). (b) Decryption computation cost (Hdr/K). (c) Total encryption and decryption cost (Hdr/K). (d) Update/revocation computation cost. (e) Total cost: encryption, decryption, and update/revocation (Hdr/K). (f) Final total computation cost: encryption and decryption of Hdr/K and message m.

For n intended receivers, SE schemes [13], [14] require the sender to encrypt the symmetric key K individually for each receiver, resulting in n encryption operations. In contrast, BE schemes [15], [16], [17], [18], [19], [20], and our scheme perform encryption only once using the public parameters of all n receivers to generate the ciphertext header, which is then broadcast to all users. This significantly reduces the number of encryption operations required at the sender side.

Through experimental evaluation, Fig. 8 provides a detailed comparison of the computational cost associated with the

TABLE II
COMPUTATIONAL COST OF EACH SCHEME FOR K/HDR

Scheme	Encryption	Key Update / Revocation	Decryption	MPK	SK
[13]	$n(T_{bp}+T_{et}+T_{sm_1}+2T_{hash})$	—	$n(T_{bp}+T_{hash})$	$\mathcal{O}(\log q)$	$\mathcal{O}(1)$
[14]	$n(T_{hash}+2T_{sm_1}+T_{pa_1})$	—	$n(4T_{sm_1}+4T_{hash}+2T_{pa_1})$	$\mathcal{O}(\log q)$	$\mathcal{O}(\log q)$
[15]	$8T_{sm_1}+(n+2)T_{pa_1}+T_{et}$	$(2n+8)T_{pa_2}+(2n+6)T_{sm_2}$	$6T_{bp}+T_{et}+5T_{pa_T}+(4n-2)T_{pa_2}+4nT_{sm_2}$	$\mathcal{O}(\log q)$	$\mathcal{O}(n\log q)$
[16]	$T_{bp}+2T_{et}+(2n+4)T_{sm_1}+(n+1)T_{pa_1}$	—	$5T_{bp}+2T_{et}+4T_{pa_T}+(4n-1)T_{sm_2}+(2n-4)T_{pa_2}$	$\mathcal{O}(n\log q)$	$\mathcal{O}(n\log q)$
[17]	$(2n+4)T_{sm_1}+2T_{et}+T_{pa_1}+T_{hash}$	—	$T_{bp}+3T_{et}+T_{pa_T}+(n+3)T_{sm_1}+(n+1)T_{pa_1}+4T_{hash}$	$\mathcal{O}(\log q)$	$\mathcal{O}(1)$
[18]	$T_{bp}+(2n+3)T_{sm_1}+2nT_{pa_1}+(3n+1)T_{hash}$	—	$T_{bp}+nT_{et}+(2n+1)T_{hash}$	$\mathcal{O}(\log q)$	$\mathcal{O}(\log q)$
[19]	$(5n+3)T_{\mathrm{sm}_1}+(2n-1)T_{\mathrm{pa}_1}$	$2nT_{\mathrm{sm}_1}+nT_{\mathrm{pa}_1}$	$nT_{\mathrm{et}}+2(n+2)T_{\mathrm{bp}}+(3n+12)T_{\mathrm{sm}_1}+(n+4)T_{\mathrm{pa}_1}+(n+2)T_{\mathrm{pa}_T}$	$\mathcal{O}(n\log q)$	$\mathcal{O}(1)$
[20]	$2T_{\mathrm{et}}+2T_{\mathrm{bp}}+(n+2)T_{sm_1}+(2n^2-n)T_{\mathrm{pa}_1}+T_{\mathrm{sm}_2}+T_{\mathrm{hash}}$	$nT_{\mathrm{sm}_1}+nT_{\mathrm{pa}_1}$	$T_{\mathrm{et}}+3T_{\mathrm{bp}}+3T_{\mathrm{sm}_1}+T_{\mathrm{pa}_1}+2T_{\mathrm{pa}_T}$	$\mathcal{O}(n\log q)$	$\mathcal{O}(1)$
Ours	$T_{sm_1}+(n+2)T_{sm_2}+T_{et}+(n+3)T_{hash}$	$2T_{sm_1}+T_{sm_2}+2T_{hash}$	$5T_{bp}+2T_{hash}+T_{et}+nT_{sm_1}+(n+3)T_{sm_2}+4T_{pa_T}$	$\mathcal{O}(\log q)$	$\mathcal{O}(\log q)$

TABLE III
COMPUTATIONAL COST OF EACH SCHEME FOR MESSAGE M

Scheme	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	Ours
Encryption	nT_{aes}	$n(T_{hash}+T_{\oplus})$	—	T_{et}	T_{et}	$T_{hash}+T_{\oplus}$	$T_{bp}+2T_{et}$	$n(T_{et}+T_{bp}+T_{pa_T}+2T_{hash})$	$T_{stream}+T_{\oplus}$
Decryption	nT_{aes}	$n(T_{hash}+T_{\oplus})$	—	T_{paT}	T_{paT}	$T_{hash}+T_{\oplus}$	$T_{et}+T_{pa_T}$	$n(T_{bp}+2T_{pa_T})$	$T_{stream}+T_{\oplus}$

TABLE IV
COMPARISON OF COMMUNICATION COSTS ACROSS ENCRYPTION SCHEMES

Scheme	Hdr	K	Ciphertext m'	MPK	SK
[13]	—	$\lvert\mathbb{G}_1\rvert+\lvert H\rvert$	$\mathcal{O}\lvert m\rvert$	$\mathcal{O}(\lambda)$	$\mathcal{O}(1)$
[14]	—	$\lvert\mathbb{G}_1\rvert$	$\mathcal{O}\lvert m\rvert$	$\mathcal{O}(\lambda)$	$\mathcal{O}(1)$
[15]	$4\lvert\mathbb{G}_1+\lvert\mathbb{Z}_p^*\rvert\rvert$	$\lvert\mathbb{G}_T\rvert$	—	$\mathcal{O}(\lambda)$	$\mathcal{O}(n\lambda)$
[16]	$3\lvert\mathbb{G}_1\rvert+2\lvert\mathbb{Z}_p^*\rvert$	$\lvert\mathbb{G}_T\rvert$	$\lvert\mathbb{G}_T\rvert$	$\mathcal{O}(\lambda)$	$\mathcal{O}(n\lambda)$
[17]	$\lvert\mathbb{G}_T\rvert+(n+3)\lvert\mathbb{G}_1\rvert$	$\lvert\mathbb{G}_T\rvert$	$\lvert\mathbb{G}_T\rvert$	$\mathcal{O}(\lambda)$	$\mathcal{O}(1)$
[18]	$2\lvert\mathbb{G}_1\rvert+(n+1)\lvert\mathbb{Z}_p^*\rvert$	$\lvert\mathbb{Z}_p^*\rvert$	$\mathcal{O}(\lvert m\rvert)$	$\mathcal{O}(\lambda)$	$\mathcal{O}(\lambda)$
[19]	$2(n+1)\lvert G_1\rvert$	$\lvert G_T\rvert$	$\lvert G_T\rvert$	$\mathcal{O}(n\lambda)$	$\mathcal{O}(1)$
[20]	$(2n+1)\lvert G_1\rvert+\lvert G_T\rvert$	$\lvert G_T\rvert$	$\lvert G_T\rvert$	$\mathcal{O}(n\lambda)$	$\mathcal{O}(\lambda)$
Ours	$\lvert\mathbb{G}_1^{cmp}\rvert+\lvert\mathbb{G}_2^{cmp}\rvert$	HK	$\lvert m\rvert$	$\mathcal{O}(\lambda)$	$\mathcal{O}(\lambda)$

Note: The Hdr, K, and ciphertext m play an important role in BE and message transmission, directly affecting the communication overhead and thus requiring precise measurement. The main public and private keys are stored in the KGC or on the vehicle, with relatively larger storage capacity. However, their storage costs are not precisely measured, and only their growth complexity is provided.

Hdr/K operations of various encryption schemes implemented in the ADS-B system, including encryption, decryption, key update/revocation, and overall computational overhead. As shown in Fig. 8(a) (encryption cost) and Fig. 8(b) (decryption cost), schemes [13], [14], and [20] incur higher costs that grow linearly or quadratically with the number of receivers n. In contrast, schemes [15], [16], [17], [18], [19], and ours demonstrate superior efficiency, with our scheme achieving consistently low computational overhead. Fig. 8(c) (total encryption and decryption cost) shows that scheme [17] attains the lowest overall cost, while our scheme performs comparably with favorable efficiency. In Fig. 8(d) (key update/revocation cost), schemes [15], [19], [20], and ours support key revocation; however, the cost of [15], [19], and [20] increases with n, whereas our scheme maintains consistently low revocation cost. Finally, Fig. 8(e) (combined cost of encryption, decryption, and key revocation) illustrates that our scheme provides full functionality with minimal total overhead, outperforming other schemes in overall computational efficiency.

Additionally, as shown in Fig. 8, from Table V, it can be seen that the encryption and decryption cost for message m in the schemes [13], [14], and [20] increases linearly with the number of receivers n, presenting an $\mathcal{O}(n)$ complexity. In contrast, schemes such as [16], [17], [18], [19], and ours show significantly lower costs for message processing. Compared to the costs for encrypting and decrypting the (Hdr/K), the overhead for encrypting and decrypting message m is relatively small. Therefore, to provide a more intuitive display of the overall computational cost, we do not plot the individual costs for message processing but instead directly present the final total computational cost, including Hdr/K computation, key update/revocation, and message computation, as shown in

TABLE V
SYSTEM-LEVEL COMPARISON OF DIFFERENT SCHEMES

Scheme	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	Ours
key update	—	—	$\mathcal{O}(\|S\|\log n)$	—	—	—	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(1)$
key revocation	—	—	$\mathcal{O}(\log n)$	—	—	—	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
identity management	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|\log n)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(\|S\|)$	$\mathcal{O}(1)$
Header re-encrypt	—	—	✗	✗	✗	✗	✓	✗	✓

Note: The key update overhead, revocation efficiency, and identity management complexity are measured by the computational load for re-encryption, the control/broadcast message complexity, and the directory lookup and aggregation cost, respectively; Header re-encrypt indicates whether the session key and header are re-encrypted during update.

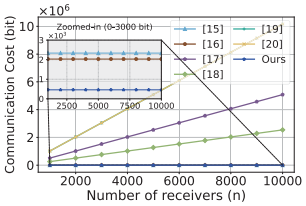

Fig. 9. Communication cost of broadcasting the header Hdr.

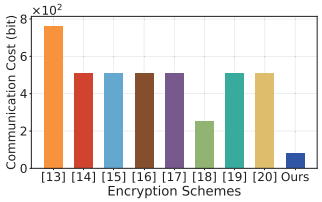

Fig. 10. Communication cost of symmetric key K.

Fig. 8(f). The results show that our ADSB-IBBE scheme has a comparable computational cost to schemes [17] and [18], but due to its support for key update/revocation, it has significant advantages in terms of flexibility and adaptability, especially for practical aviation scenarios requiring dynamic key management. Moreover, Fig. 8 also demonstrates the scalability of our scheme, as the computation cost grows slowly with the number of receivers and remains acceptable even at 10 000 users, confirming its suitability for large-scale deployments. Although it performs well in terms of computational cost, we still need to further analyze the overall performance of the schemes by considering communication costs to more comprehensively assess their applicability, especially given the relatively small message size constraint in ADS-B systems.

B. Communication Cost

In addition to computational cost, we also evaluate the communication cost of different encryption schemes, focusing on the bandwidth consumption during data transmission. Specifically, we assess the size of the symmetric key K, the encrypted header Hdr, and the message ciphertext m, both of which directly impact the communication overhead required for message transmission. Furthermore, we also present the storage complexity for the MPK and the private key.

Table IV provides a detailed comparison of the communication overhead between our ADSB-IBBE scheme and other encryption solutions. Our scheme employs point compression techniques, reducing the size of the header Hdr′ to $|\mathbb{G}_1^{\text{cmp}}| + |\mathbb{G}_2^{\text{cmp}}|$ data elements, significantly lowering the transmission overhead. In contrast, other encryption schemes, such as scheme [15] requires $4|\mathbb{G}_1| + |\mathbb{Z}_p^*|$, and scheme [17] needs $|\mathbb{G}_T| + (n+3)|\mathbb{G}_1|$, making our scheme more lightweight in terms of data transmission.

To calculate the communication costs of these encryption schemes, we first determine the bit size of each element. The sizes of $|\mathbb{G}_1|$, $|\mathbb{G}_2|$, and $|\mathbb{G}_T|$ elements are 508 bits, while the size of $|\mathbb{Z}_p^*|$ elements is 254 bits. Based on these sizes, we can compute the communication costs for the Hdr, K, and ciphertext message m'. Figs. 9 and 10 illustrate the corresponding communication costs in bits. Since Hdr needs to be broadcast in real-time, its communication cost directly impacts bandwidth consumption. To better highlight the differences between schemes [15], [16], and Ours, a local zoomed-in view is provided in Fig. 9 at the 0–3000-bit range, where it is evident that our scheme occupies the least data bits for broadcasting Hdr, significantly outperforming the others. Although K does not require broadcast transmission, its storage still impacts the system's encryption/decryption management overhead. Fig. 10 shows that our scheme requires only 80 bits for K, significantly reducing its storage overhead.

As shown in Table IV, the length of the ciphertext m' for schemes [13], [14], and [18] remains proportional to the length of plaintext m, i.e., $\mathcal{O}(|m|)$, with m' being at most proportional to the size of m, possibly with additional overhead such as padding or extra tags. In contrast, schemes [16], [17], [19], and [20] have a ciphertext length of $|G_T|$, which is 508 bits, generally larger than the plaintext length, resulting in increased storage and transmission costs and higher communication overhead. In comparison, our scheme ensures that the ciphertext length is exactly equal to the plaintext length. Specifically, we encrypt the AA and ME fields of the ME, which are 80 bits, making the ciphertext length precisely 80 bits, leading to minimal communication overhead without additional costs. As the ADS-B system requires broadcasting both the Hdr and m', our scheme offers the lowest communication cost and a significant advantage, which is crucial for ADS-B systems often constrained by low bandwidth and limited message data bits.

In summary, our scheme strikes the optimal balance between computational cost, communication cost, and applicability. Although scheme [17] offers lower computational overhead, its Hdr length increases with the number of users, which is impractical in ADS-B BE (a similar issue exists in scheme [18]). In contrast, our scheme has a fixed Hdr size of only 510 bits and a ciphertext size of just 80 bits, ensuring the

lowest communication overhead while keeping computational cost low.

Beyond computational and communication analyses, we further evaluate our scheme from a system-level perspective. Specifically, key update overhead is measured by the overall computational load required for re-encryption of a new broadcast set; revocation efficiency is evaluated by the number or complexity of control/broadcast messages during the revocation process; and identity management complexity is measured by the cost of directory query and aggregation. In addition, the Header re-encrypt indicates whether the session key and header are re-encrypted during update. The comparison results are summarized in Table V, showing that our scheme achieves excellent performance across all four dimensions, demonstrating that it not only minimizes encryption overhead but also maintains system-level scalability and efficiency.

Moreover, the ADSB-IBBE framework benefits from its underlying consortium-based sharding blockchain, which ensures scalable and lightweight identity management. Together, these features reinforce the practicality of the proposed scheme for future large-scale aviation networks and establish it as a well-balanced and efficient solution for secure ADS-B BE.

VI. Conclusion

This study proposes a blockchain-based ADSB-IBBE scheme specifically designed for large-scale aerial vehicle communication, enabling decentralized identity management and key revocation to enhance both security and scalability. By employing a lightweight IBBE construction, the scheme achieves multireceiver key sharing while reducing overhead, ensuring efficiency in high-frequency broadcast environments. Decentralized key management with a sharding blockchain eliminates reliance on centralized trust and mitigates the risk of SPOF. Furthermore, stream cipher encryption with a ciphertext length equal to the plaintext ensures compatibility with the stringent constraints of ADS-B. Experimental results confirm that the scheme achieves an optimal balance among computational cost, communication overhead, and key management efficiency, making it suitable for diverse aviation scenarios such as civil aviation, UAVs, and eVTOL. In addition, the scheme enhances the security and stability of ATM systems in high-frequency broadcast environments and establishes a reliable foundation for the digital communication security of NextGen ATM, supporting collaborative airspace management for both crewed aircraft and UAVs. Future work will focus on blockchain-induced limitations in throughput, high latency, and scalability issues [11], [12], aiming to enhance performance through the use of multileader BFT.

Appendix A
Security Proof of ADSB-IBBE

Theorem 2: Let the cryptographic hash functions H_1 and H_2 be random oracles. If the (f, g)-GDDHE assumption holds, the identity-based BE scheme presented in this article is IND-sID-CCA secure.

The (f, g)-GDDHE assumption is particularly suitable for ADS-B environments involving identity-based BE with dynamic receiver updates and key revocation. It captures the complexity of managing multiple receivers with evolving identity sets.

To ensure the clarity and efficiency of the security proof, we analyze the scheme under the ROM. Although the proof of security based on the (f, g)-GDDHE assumption and IND-sID-CCA security is theoretically complex, by idealizing the hash functions as random oracles, we are able to simplify the handling of hash functions and focus on proving the security of the scheme under these assumptions. This approach has been widely adopted in classical IBE and IBBE constructions, and its reasonableness has been validated both theoretically and practically. Although ROM may not accurately capture real-world scenarios, our domain-separated hash-to-field design ensures statistically near-uniform outputs, cross-purpose independence, and canonical input consistency, making the practical implementation closely approximate the behavior assumed in the ROM. Therefore, the adoption of ROM as the formal security model in this scheme is well justified.

Proof: Suppose there exists an adversary $\mathcal{A}$ that can break the IND-sID-CCA security of the proposed scheme with a nonnegligible advantage ϵ. We construct a simulator $\mathcal{B}$ that interacts with $\mathcal{A}$ and solves the (f, g)-GDDHE problem with a nonnegligible advantage.

The ADS-B BE system handles up to l users, with a total of τ queries. The simulator $\mathcal{B}$ is given an instance of the (f, g)-GDDHE problem

$$g_0, g_0^{\alpha}, \ldots, g_0^{\alpha^{\tau-1}}, g_0^{\alpha f(\alpha)}, g_0^{k\alpha f(\alpha)}, g_0^{tf(\alpha)}, g_0^{\alpha t a_i f(\alpha)}$$
$$h_0, h_0^{\alpha}, \ldots, h_0^{\alpha^{2l+8}}, h_0^{a_i t}, h_0^{\alpha k g(\alpha)}$$

Let T be an element of the bilinear group G_T. The task is to determine whether $T = e(g_0, h_0)^{k\alpha f(\alpha)(\alpha - x_{\tau+l+1})(\alpha - t)}$ or whether T is a random element in G_T.

The functions $f(x)$ and $g(x)$ are coprime polynomials of degrees τ and l, respectively, given by

$$f(x) = \prod_{i=1}^{\tau}(x - x_i), \quad g(x) = \prod_{i=\tau+1}^{\tau+l+2}(x - x_i). \tag{27}$$

Define $f_i(x) = (f(x))/(x - x_i)$ for $i \in [1, \tau]$ and $g_i(x) = (g(x))/(x - x_i)$ for $i \in [\tau + 1, \tau + l + 2]$.

Initialization Phase: The adversarial algorithm $\mathcal{A}$ outputs a set of challenge identities $S^* = \{\mathsf{ID}_1^*, \mathsf{ID}_2^*, \ldots, \mathsf{ID}_{s^*}^*\}$, where $s^* \le l$.

System Setup Phase: To generate the system MPK, the simulator $\mathcal{B}$ secretly sets $g = g_0^{f(\alpha)}$, without knowing the value of g. Then, $\mathcal{B}$ sets the message header Hdr with $C_1^* = g_0^{k\alpha f(\alpha)}$, and other parameters are defined as follows:

$$h = h_0^{\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha - x_i)}, \quad g^{\alpha} = g_0^{\alpha f(\alpha)} \tag{28}$$

$$g_T = e(g_0, h_0)^{f(\alpha)\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha - x_i)}. \tag{29}$$

The key derivation function $HKDF : \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T \times \{0,1\}^* \to \{0,1\}^{\mathrm{hl}}$ is then applied to compute the system MPK, which is sent to $\mathcal{A}$.

$$\mathsf{MPK} = (g^{\alpha}, g^{\alpha^l}, g_T, h, h^t, h^{\alpha}, \ldots, h^{\alpha^{l+2}}, HKDF). \tag{30}$$

Hash Query Phase: At any point, the adversary $\mathcal{A}$ may query the hash functions H_1 or H_2 on an identity ID_i. Suppose $\mathcal{A}$ makes at most e extraction queries and q decryption queries, then the number of hash queries is at most $\tau-e-q$. To simulate these, the simulator $\mathcal{B}$ maintains a hash list $\mathcal{LH}$, where each entry is of the form $(u, \mathsf{ID}_i, x_i, \mathsf{SK}_{\mathsf{ID}_i})$, with $u \in \{1,2\}$ indicating a query to H_1 or H_2. The list $\mathcal{LH}$ is initialized as follows:

$$\{(\perp,\perp,x_i,\perp)\}_{i\in[1,\tau]}, \{(1,\mathsf{ID}_i,x_i,\perp)\}_{i\in[\tau+1,\tau+s^*]}$$
$$\{(\perp,\perp,x_i,\perp)\}_{i\in[\tau+s^*+1,\tau+l]}$$
$$\{(2,C_1^*,x_i,\perp)\}_{i=\tau+l+1}, \{(2,\perp,x_i,\perp)\}_{i=\tau+l+2}$$

When the adversary $\mathcal{A}$ makes a hash query on identity ID_i to H_1, the simulator $\mathcal{B}$ responds as

1) If $(1, \mathsf{ID}_i, x_i, \perp) \in \mathcal{LH}$, return x_i.
2) Otherwise, $\mathcal{B}$ sets $H_1(\mathsf{ID}_i) = x_i$, stores $(1, \mathsf{ID}_i, x_i, \perp)$ in $\mathcal{LH}$, and returns x_i.

When $\mathcal{A}$ makes a hash query on identity ID_i to H_2, $\mathcal{B}$ responds as

1) If $(2, \mathsf{ID}_i, x_i, \perp) \in \mathcal{LH}$, return x_i.
2) Otherwise, $\mathcal{B}$ sets $H_2(\mathsf{ID}_i) = x_i$, stores $(2, \mathsf{ID}_i, x_i, \perp)$ in $\mathcal{LH}$, and returns x_i.

When $\mathcal{A}$ makes an H_2 query on a private key $\mathsf{SK}_{\mathsf{ID}_i} = (K_{1,i}, K_{2,i}, K_{3,i})$, where $\mathsf{SK}_{\mathsf{ID}_i}$ is either obtained through an extraction query or randomly generated by $\mathcal{A}$, and $\mathsf{ID}_i \notin S^*$, the simulator $\mathcal{B}$ proceeds as

1) If $(\perp, \perp, x_i, \mathsf{SK}_{\mathsf{ID}_i}) \in \mathcal{LH}$, return x_i.
2) Otherwise, $\mathcal{B}$ sets $H_2(\mathsf{SK}_{\mathsf{ID}_i}) = x_i$, stores $(2, \mathsf{ID}_i, x_i, \mathsf{SK}_{\mathsf{ID}_i})$ in $\mathcal{LH}$, and returns x_i.

Query Phase 1: The adversary $\mathcal{A}$ may adaptively issue extraction queries and decryption queries to the simulator $\mathcal{B}$.

1) *Extraction Query:* When the adversary $\mathcal{A}$ queries the private key corresponding to an identity $\mathsf{ID}_i \notin S^*$, the simulator $\mathcal{B}$ searches the list $\mathcal{LH}$ for an entry of the form $(1, \mathsf{ID}_i, x_i, \mathsf{SK}_{\mathsf{ID}_i})$. If found, it returns the corresponding private key $\mathsf{SK}_{\mathsf{ID}_i}$ to $\mathcal{A}$. If an entry $(1, \mathsf{ID}_i, x_i, \perp)$ exists, $\mathcal{B}$ directly uses the stored x_i. If no such entry exists, $\mathcal{B}$ performs a hash query on ID_i to obtain x_i, and then computes the private key $\mathsf{SK}_{\mathsf{ID}_i}$

$$K_{1,i} = g_0^{\alpha a_i t f(\alpha)}$$
$$K_{2,i} = h_0^{-\alpha^2 a_i t(\alpha-x_i)\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i)}$$
$$K_{3,i} = g_0^{\frac{f(\alpha)(\alpha-t)}{\alpha-x_i}} = g_0^{f_i(\alpha)(\alpha-t)}. \quad (31)$$

The simulator $\mathcal{B}$ then stores $(1, \mathsf{ID}_i, x_i, \mathsf{SK}_{\mathsf{ID}_i})$ in $\mathcal{LH}$ and returns $\mathsf{SK}_{\mathsf{ID}_i} = (K_{1,i}, K_{2,i}, K_{3,i})$ to $\mathcal{A}$.

2) *Decryption Query:* When $\mathcal{A}$ submits a decryption query on a ciphertext header $\mathsf{Hdr} = (C_1, C_2)$ under a target identity subset $S \subseteq S^*$, the simulator $\mathcal{B}$ retrieves the entries $(\perp, \mathsf{ID}_i, x_i, \mathsf{SK}_{\mathsf{ID}_i})$ from $\mathcal{LH}$ for each $\mathsf{ID}_i \in S$. For any valid header Hdr, the simulator $\mathcal{B}$ implicitly sets $S^* \cup \{C_1^*\}$, and searches for $(2, C_1, x_i, \hat{\mathsf{SK}}_{C_1})$ in the list $\mathcal{LH}$. If not found, $\mathcal{B}$ performs a hash query to H_2 on C_1 to compute $(2, C_1, x_i, \hat{\mathsf{SK}}_{C_1})$, where $\hat{\mathsf{SK}}_{C_1} = (\hat{K}_{1,i}, \hat{K}_{2,i}, \hat{K}_{3,i})$

$$\hat{K}_{1,i} = g_0^{\alpha a_i t f(\alpha)}$$
$$\hat{K}_{2,i} = h_0^{-\alpha^2 a_i t(\alpha-x_i)\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i)}$$
$$\hat{K}_{3,i} = g_0^{f_i(\alpha)(\alpha-t)}. \quad (32)$$

The simulator $\mathcal{B}$ uses this to simulate decryption and returns the corresponding symmetric key K to $\mathcal{A}$.

Challenge Phase: When the adversary $\mathcal{A}$ completes Query Phase 1, it outputs a ciphertext header $\mathsf{Hdr}^* = (C_1^*, C_2^*)$ associated with a valid identity $\mathsf{ID}_i \in S^*$, along with a symmetric key K

$$C_1^* = g_0^{k\alpha f(\alpha)}, \quad C_2^* = h_0^{\alpha k(\alpha-x_{\tau+l+1})g(\alpha)}. \quad (33)$$

If $T = e(g_0, h_0)^{k\alpha f(\alpha)(\alpha-x_{\tau+l+1})(\alpha-t)}$ then

$$\begin{aligned}(\varpi^*)^{\prod_{i=\tau+1}^{\tau+s^*}x_i} &= \frac{T^{\prod_{i=\tau+1}^{\tau+l+2}(\alpha-x_i)}\cdot e(g_0^{k\alpha f(\alpha)}, h_0^{\Theta})}{e(g_0,h_0)^{kf(\alpha)x_{\tau+l+2}\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i)\prod_{i=\tau+1}^{\tau+s^*}x_i}} \\ &= \frac{e(g_0,h_0)^{kf(\alpha)x_{\tau+l+2}\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i)\prod_{i=\tau+1}^{\tau+s^*}x_i}}{e(g_0,h_0)^{kf(\alpha)x_{\tau+l+2}\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i)\prod_{i=\tau+1}^{\tau+s^*}x_i}} \\ \Theta=\prod_{i=\tau+s^*+1}^{\tau+l+2}(\alpha-x_i) &\cdot\Big[(t-\alpha)(\alpha-x_{\tau+l+1})\prod_{i=\tau+1}^{\tau+s^*+1}(\alpha-x_i)+x_{\tau+l+2}\prod_{i=\tau+1}^{\tau+s^*}x_i/\alpha\Big]. \quad (34)\end{aligned}$$

Thus

$$\begin{aligned}\varpi^* &= \frac{e(g,h)^{kH_2(C_1)}}{e(g,h)^{kH_2(K_{1,i}\|K_{2,i}\|K_{3,i})}} \quad (35)\\ &= e(g,h)^{k(H_2(C_1)-H_2(K_{1,i}\|K_{2,i}\|K_{3,i}))} \\ K^* &= \mathrm{HKDF}(C_1, C_2, \varpi^*, S \parallel l; hl) \quad (36)\end{aligned}$$

The simulator $\mathcal{B}$ samples $\mu \leftarrow_R \{0,1\}$, sets $K_\mu = K^*$, and selects $K_{1-\mu} = (K^*)'$ uniformly at random from the key space $\mathcal{K}$. It then returns $(\mathsf{Hdr}^*, K_0, K_1)$ to the adversary $\mathcal{A}$.

Query Phase 2: The adversary A continues to issue extraction and decryption queries. The simulator B responds in the same strategy as in Query Phase 1. However, note that decryption queries must satisfy the restriction $\mathsf{Hdr} \neq \mathsf{Hdr}'$.

Guess Phase: Eventually, the adversary $\mathcal{A}$ outputs a guess $\mu' \in \{0,1\}$. If $\mu = \mu'$, this implies that $T = e(g_0, h_0)^{krf(r)}$, and $\mathcal{A}$ wins the game. Otherwise, if T is a random element in the group $\mathbb{G}_T$, $\mathcal{A}$ does not win the game

$$\begin{aligned}&\mathsf{Adv}_{\mathcal{B}}^{(f,g)\text{–GDDHE}} \\ &= \Pr[\mu=\mu' \mid T = e(g_0,h_0)^{k\alpha f(\alpha)}] \\ &\quad - \Pr[\mu=\mu' \mid T \in_{\mathcal{R}} \mathbb{G}_T] \\ &= \left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND\text{–}sID\text{–}CCA}} + \frac{1}{2} - \Pr[\mu=\mu' \mid T \in_{\mathcal{R}} \mathbb{G}_T]\right| \\ &= \left|\epsilon + \frac{1}{2} - \frac{1}{2}\right| = \epsilon. \quad (37)\end{aligned}$$

□

Appendix B
System Security Analysis

We analyze the security of the proposed ADSB-IBBE scheme based on its design goals and a realistic threat model, where adversaries may passively eavesdrop on open broadcasts, actively inject or forge ME, and act as compromised or revoked insiders.

Confidentiality: The scheme employs IBBE, ensuring that only authorized receivers in the broadcast identity set can derive the same symmetric key K. Even if an adversary obtains the ciphertext and all public parameters, it remains computationally infeasible to recover the plaintext without a valid private key. Our scheme is formally proven to satisfy IND-sID-CCA security under the (f, g)-GDDHE assumption in the ROM.

Identity Privacy: The encrypted ME conceals both sender and receiver identities. Compressed ciphertext headers and concealed broadcast identity sets prevent adversaries from inferring who can decrypt the message, ensuring recipient anonymity. Meanwhile, the sender's identity AA field is encrypted via a stream cipher, achieving indistinguishability of identities and protecting broadcaster privacy. These mechanisms collectively mitigate tracking and inference attacks.

Revocability and Updatability: The scheme supports dynamic revocation and secure key updates. When a user's key is compromised or their authorization status changes, the system can promptly revoke their decryption rights by updating the private keys. Revocation information is propagated via the blockchain, ensuring that revoked users cannot access future encrypted content. Even if a partial key is leaked, it alone is insufficient for decryption, and key updates or revocations are still required.

Decentralized Identity Management: The scheme integrates a shard-based blockchain to enable decentralized and tamper-resistant storage of identity and revocation data. This design mitigates SPOF in traditional centralized infrastructures. The blockchain ensures that all aerial vehicles and ground stations have access to the most up-to-date BE parameters.

Message Structure Compatibility: A lightweight stream cipher is used to encrypt the AA and ME fields of the ME. The ciphertext maintains the original message length and format, preserving compatibility with the ADS-B protocol and requiring no modification to the transmission structure. This ensures real-time performance under constrained bandwidth conditions.

Attack Resistance: The proposed scheme defends against common threats, including replay attacks, ciphertext forgery, collusion among revoked users, key leakage, delay attacks, and DDoS attacks.

1) *Replay Attacks:* The use of timestamps and random values such as k, ensures the uniqueness of each message, preventing the reuse of old ciphertexts.
2) *Ciphertext Forgery:* The use of dynamically generated keystreams and hash functions ensures that each encryption produces a distinct ciphertext, preventing forgery.
3) *Key Leakage:* IBBE with key revocation ensures that leaked keys cannot decrypt future messages, as revoked keys are quickly invalidated.
4) *Collusion Among Revoked Users:* The $\mathcal{RL}$ and decentralized identity management prevent revoked users from decrypting messages, even if they collude, as their keys are invalidated.
5) *Delay Attacks:* Timestamps and candidate timestamps $\mathrm{ts}_{\mathsf{candi}}$ allow the receiver to handle minor time discrepancies, discarding messages outside the acceptable time window.
6) *DDoS Attacks:* The decentralized design mitigates DDoS risks by preventing SPOF.

REFERENCES

[1] T. Bolić and P. Ravenhill, "SESAR: The past, present, and future of European air traffic management research," *Engineering*, vol. 7, no. 4, pp. 448–451, Apr.2021.

[2] Cirium, *2024-2043 Cirium Fleet Forecast*. London, U.K.: Cirium Ascend Consultancy, 2024.

[3] *Faa Aerospace Forecast Fiscal Years 2024-2044*, Federal Aviation Administration, Washington, DC, USA, 2024.

[4] D. Kožović, D. Ž. Đurđević, M. Dinulović, S. Milić, and B. Rašuo, "Air traffic modernization and control: ADS-B system implementation update 2022: A review," *FME Trans.*, vol. 51, no. 1, pp. 117–130, 2023.

[5] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up OpenSky: A large-scale ADS-B sensor network for research," in *Proc. 13th Int. Symp. Inf. Process. Sensor Netw.*, Apr. 2014, pp. 83–94.

[6] C. Yao, X. Zhang, Y. Liu, B. Zhao, Q. Wu, and W. Susilo, "Blockchain-based secure and efficient ADS-B authentication via certificateless signature with packet loss tolerance," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10574–10588, Apr.2025.

[7] Z. Wu, T. Shang, and A. Guo, "Security issues in automatic dependent surveillance–broadcast (ADS-B): A survey," *IEEE Access*, vol. 8, pp. 122147–122167, 2020.

[8] B. S. Ali, W. Y. Ochieng, W. Schuster, A. Majumdar, and T. K. Chiew, "A safety assessment framework for the automatic dependent surveillance broadcast (ADS-B) system," *Saf. Sci.*, vol. 78, pp. 91–100, Oct.2015.

[9] H. Khan, H. Khan, S. Ghafoor, and M. A. Khan, "A survey on security of automatic dependent surveillance-broadcast (ADS-B) protocol: Challenges, potential solutions, and future directions," *IEEE Commun. Surveys Tuts.*, vol. 27, pp. 3199–3226, 2024.

[10] Y. Liu et al., "A flexible sharding blockchain protocol based on cross-shard Byzantine fault tolerance," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2276–2291, 2023.

[11] Y. Liu et al., "Kronos: A secure and generic sharding blockchain consensus with optimized overhead," in *Proc. NDSS*, 2025.

[12] A. Liu et al., "CHERUBIM: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3178–3193, 2024.

[13] J. Baek, E. Hableel, Y.-J. Byon, D. S. Wong, K. Jang, and H. Yeo, "How to protect ADS-B: Confidentiality framework and efficient realization based on staged identity-based encryption," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 3, pp. 690–700, Mar.2017.

[14] A. Braeken, "Holistic air protection scheme of ADS-B communication," *IEEE Access*, vol. 7, pp. 65251–65262, 2019.

[15] A. Ge and P. Wei, "Identity-based broadcast encryption with efficient revocation," in *Proc. 22nd IACR Int. Conf. Pract. Theory Public-Key Cryptogr.*, Beijing, China, Apr. 2019, pp. 405–435.

[16] S. C. Ramanna, "More efficient constructions for inner-product encryption," in *Proc. 14th Int. Conf. Appl. Cryptogr. Netw. Security*, Jun. 2016, pp. 231–248.

[17] L. Chen, J. Li, Y. Lu, and Y. Zhang, "Adaptively secure certificate-based broadcast encryption and its application to cloud storage service," *Inf. Sci.*, vol. 538, pp. 273–289, Oct.2020.

[18] L. Deng, "Anonymous certificateless multi-receiver encryption scheme for smart community management systems," *Soft Comput.*, vol. 24, no. 1, pp. 281–292, Jan.2020.

[19] J. Chen, J. Niu, H. Lei, L. Lin, and Y. Ling, "Adaptively secure multi-authority attribute-based broadcast encryption in fog computing," *Comput. Netw.*, vol. 232, Aug.2023, Art. no. 109844.

[20] S. Liu, H. Pan, X. A. Wang, S. Zhao, and Q. Li, "A traceable and revocable broadcast encryption scheme for preventing malicious encryptors in medical IoT," *J. Syst. Archit.*, vol. 149, Apr.2024, Art. no. 103100.

[21] Y. Rong et al., "Semantic entropy can simultaneously benefit transmission efficiency and channel security of wireless semantic communications," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2067–2082, 2025.

[22] X. Cao et al., "Exploring LLM-based multi-agent situation awareness for zero-trust space-air-ground integrated network," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 2230–2247, Jun.2025.

[23] H. Lu et al., "Malsight: Exploring malicious source code and benign pseudocode for iterative binary malware summarization," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 6733–6747, 2025.

[24] C. Qiu et al., "Disentangled dynamic intrusion detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 11, pp. 10528–10545, Nov.2025.

[25] H. Yang, Q. Zhou, M. Yao, R. Lu, H. Li, and X. Zhang, "A practical and compatible cryptographic solution to ADS-B security," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3322–3334, Apr.2019.

[26] F. Zeng, "Secure ads-b protection scheme supporting query," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trusted Computing, Scalable Comput. Commun., Internet People Smart City Innov.*, Oct. 2021, pp. 513–518.

[27] B. Burfeind, R. Mills, S. Nykl, J. A. Betances, and C. Sielski, "Confidential ADS-B," in *Proc. IEEE Aerosp. Conf.*, Mar. 2019, pp. 1–11.

[28] A. Fiat and M. Naor, "Broadcast encryption," in *Proc. Annu. Int. Cryptol. Conf.*, Jan. 1993, pp. 480–491.

[29] D. Boneh, C. Gentry, and B. Waters, "Collusion resistant broadcast encryption with short ciphertexts and private keys," in *Proc. 25th Annu. Conf. Adv. Cryptol. (CRYPTO)*, vol. 3621. Cham, Switzerland: Springer, 2005, pp. 258–275.

[30] D. Guo, Q. Wen, W. Li, H. Zhang, and Z. Jin, "Adaptively secure broadcast encryption with constant ciphertexts," *IEEE Trans. Broadcast.*, vol. 62, no. 3, pp. 709–715, Sep.2016.

[31] S. Agrawal, D. Wichs, and S. Yamada, "Optimal broadcast encryption from LWE and pairings in the standard model," in *Proc. 18th Int. Conf. Theory Cryptography*, Durham, NC, USA, Nov. 2020, pp. 149–178.

[32] A. Dupin and S. Abelard, "Broadcast encryption using sum-product decomposition of Boolean functions," *IACR Commun. Cryptol.*, vol. 1, no. 1, 2024.

[33] C. Delerablée, "Identity-based broadcast encryption with constant size ciphertexts and private keys," in *Proc. 13th Int. Conf. Theory Appl. Cryptol. Inf. Secur. Adv. Cryptol.*, 2007, pp. 200–215.

[34] C. Gentry, "Practical identity-based encryption without random oracles," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2006, pp. 445–464.

[35] B. Waters, "Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions," in *Proc. Annu. Int. Cryptol. Conf.*, 2009, pp. 619–636.

[36] C. Gentry and B. Waters, "Adaptive security in broadcast encryption systems (with short Ciphertexts)," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2009, pp. 171–188.

[37] J. Kim, W. Susilo, M. H. Au, and J. Seberry, "Adaptively secure identity-based broadcast encryption with a constant-sized ciphertext," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 3, pp. 679–693, Mar.2015.

[38] M. Mandal, "Privacy-preserving fully anonymous ciphertext policy attribute-based broadcast encryption with constant-size secret keys and fast decryption," *J. Inf. Secur. Appl.*, vol. 55, Dec.2020, Art. no. 102666.

[39] J. Zhang, S. Su, H. Zhong, J. Cui, and D. He, "Identity-based broadcast proxy re-encryption for flexible data sharing in VANETs," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4830–4842, 2023.

[40] H. Zhong, S. Zhang, J. Cui, L. Wei, and L. Liu, "Broadcast encryption scheme for V2I communication in VANETs," *IEEE Trans. Veh. Technol.*, vol. 71, no. 3, pp. 2749–2760, Mar.2022.

[41] M. Strohmeier, V. Lenders, and I. Martinovic, "On the security of the automatic dependent surveillance-broadcast protocol," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 2, pp. 1066–1087, 2nd Quart., 2015.

[42] C. De Cannière, "TRIVIUM: A stream cipher construction inspired by block cipher design principles," in *Proc. Int. Conf. Inf. Secur.*Cham, Switzerland: Springer, 2006, pp. 171–186.

[43] *Mode S Downlink Aircraft Parameters Implementation and Operations Guidance Document*, 5thed., Bangkok, Thailand: International Civil Aviation Organization (ICAO), Asia and Pacific Office, Mar.2023.

Xuejun Zhang (Member, IEEE) received the B.S. and Ph.D. degrees from the School of Electronic and Information Engineering, Beihang University, Beijing, China, in 1994 and 2000, respectively.

He is currently a Professor with the School of Electronic and Information Engineering, Beihang University. His main research interests are aviation data communication, integrated air surveillance, uncrewed aerial vehicles, and intelligent air traffic management.

Chong Yao received the M.S. degree from China University of Mining and Technology, Xuzhou, China, in 2020. She is currently pursuing the Ph.D. degree with Beihang University, Beijing, China.

Her research interests include air traffic management, aviation data security, air surveillance, uncrewed aerial vehicles, cryptography, blockchain, and AI security.

Yizhong Liu (Member, IEEE) received the B.S. and Ph.D. degrees in electronic and information engineering from Beihang University, Beijing, China, in 2014 and 2021, respectively.

He is now an Associate Professor at the School of Cyber Science and Technology, Beihang University. He has authored over 60 publications, including ACM CCS, NDSS, USENIX Security, S&P, WWW, CVPR, ACM MM, ICCV, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, etc. His research interests include cryptography, blockchain, and AI security.

Boyu Zhao received the B.S. degree in information security from Beihang University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology.

His research interests include cryptography and blockchain consensus.

Cheng Chi received the master's degree from the University of Sheffield, Sheffield, U.K., in 2015.

He is currently a Researcher with the Research Institute of Industrial Internet of Things, China Academy of Information and Communications Technology, Beijing, China. His research interests include Industrial Internet resolution systems and blockchain information services.

Haohua Du (Member, IEEE) received the M.S. and Ph.D. degrees from the Department of Computer Science, Illinois Institute of Technology, Chicago, IL, USA, in 2015 and 2019, respectively.

She is currently an Assistant Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. Her research interests include wireless sensing, wireless communication, and other IoT applications.